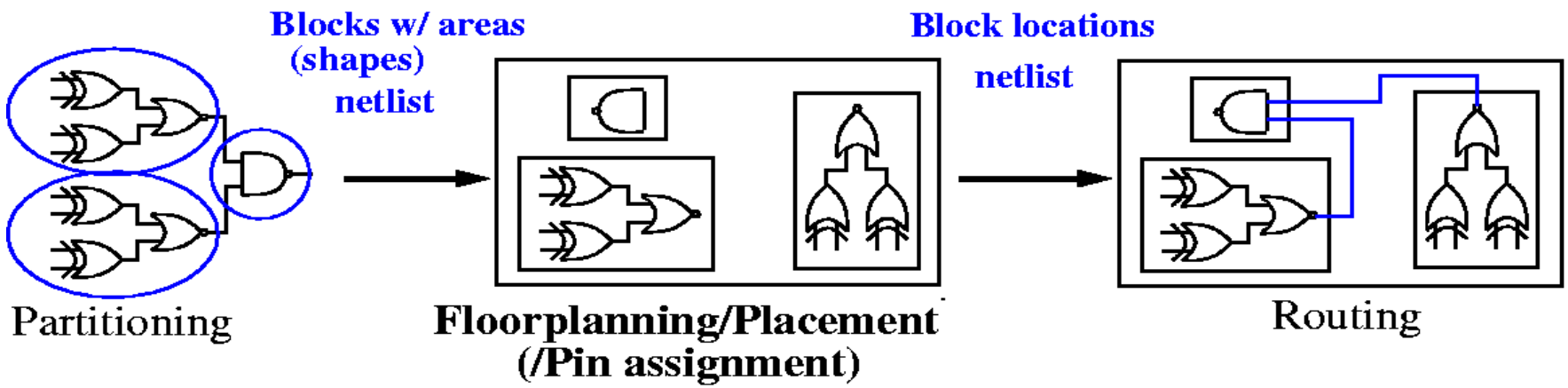
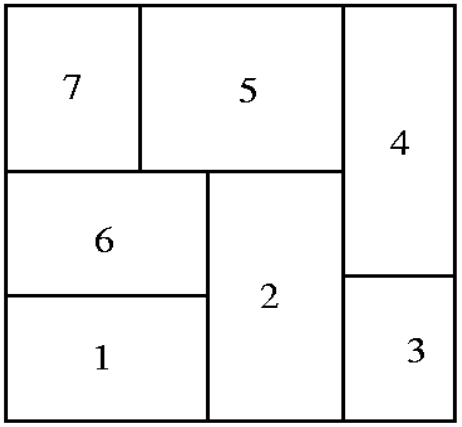
Floorplanning

- Partitioning leads to
  - Modules (or called blocks) with well-defined areas and shapes (*hard modules*).
  - Modules with approximated areas and no particular shapes (*soft modules*).
  - A netlist specifying connections between the modules.
- Objectives
  - Find **locations** for all modules, as well as **orientations (if allowable)** for hard modules.
  - Find shapes (and pin locations) of the soft modules.

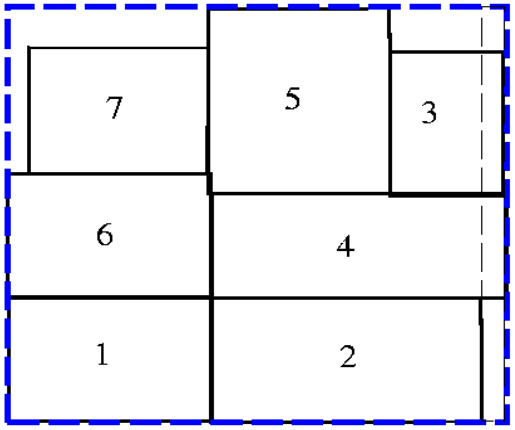


Floorplanning Problem

- Inputs:
  - A set of modules, hard or soft.
  - Pin locations of hard modules.
  - A netlist.
- Objectives: Minimize area, reduce wirelength for (critical) nets, maximize routability (minimize congestion), determine shapes of soft modules.



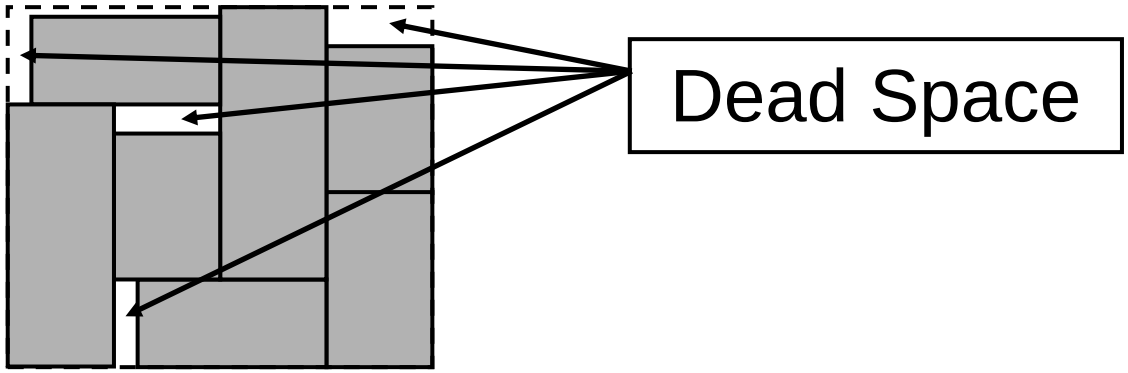
An optimal floorplan, in terms of area



A non–optimal floorplan

Dead Space

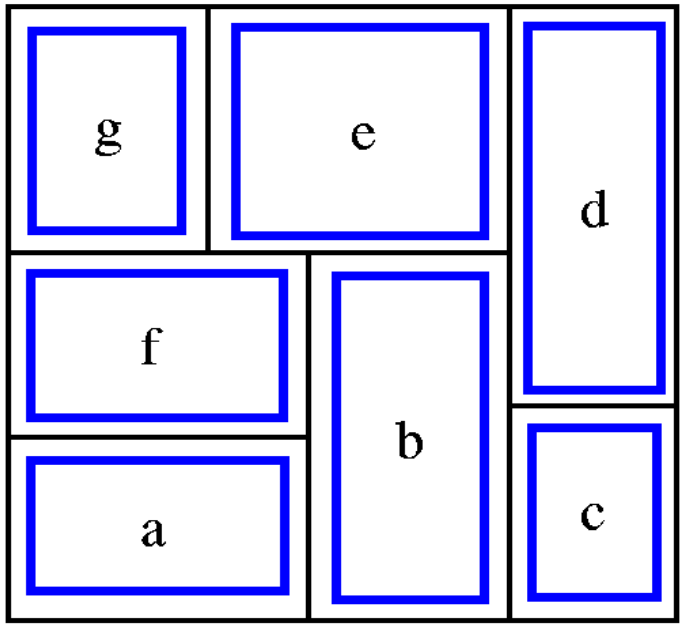
- The space that is wasted.

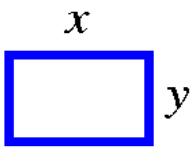


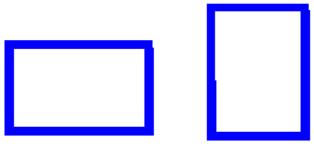
- Minimizing area is the same as minimizing dead space.
- Percentage of dead space

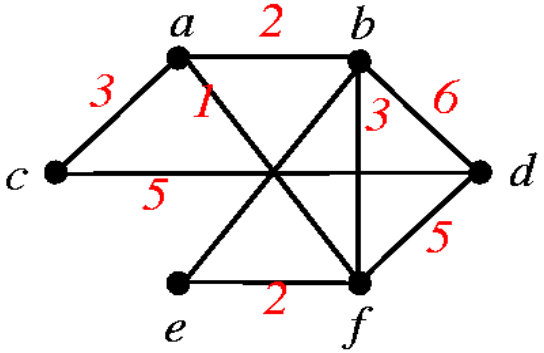
  = ((Area of resulting rectangle / Total area of all modules) – 1) x 100%.

Floorplan Design



- *Modules:* 

- *Area: A=xy*
- *Aspect ratio: r <= y/x <= s*
- *Rotation:* 
- *Module connectivity*



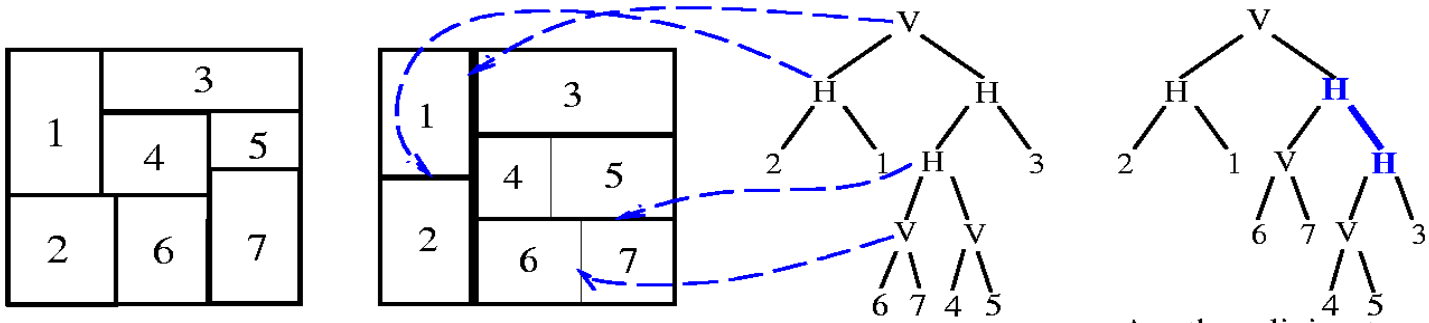
Bounds on Aspect Ratios

- If there is no bounds on aspect ratios, we can always pack modules completely tight (i.e., no dead space).

- We do not want to layout a module as a long strip.

Terminology

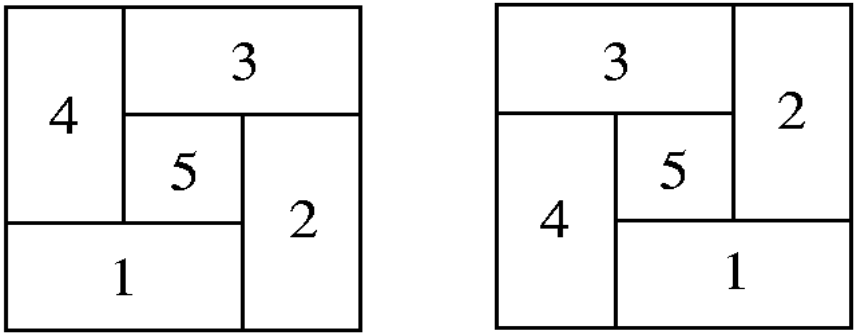
- **Rectangular dissection**: Subdivision of a given rectangle by a finite # of horizontal and vertical line segments into a finite # of non-overlapping rectangles.
- **Slicing structure**: a rectangular dissection that can be obtained by repetitively subdividing rectangles horizontally or vertically.
- **Slicing tree**: A binary tree, where each internal node represents a vertical cut line or horizontal cut line, and each leaf a basic rectangle.
- **Skewed slicing tree**: A slicing tree in which no node and its right child are the same type of cut line.



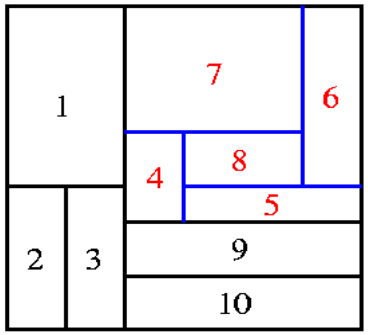
Non−slicing floorplan Slicing floorplan A slicing tree (skewed) Another slicing tree (non−skewed)

Terminology (cont'd)

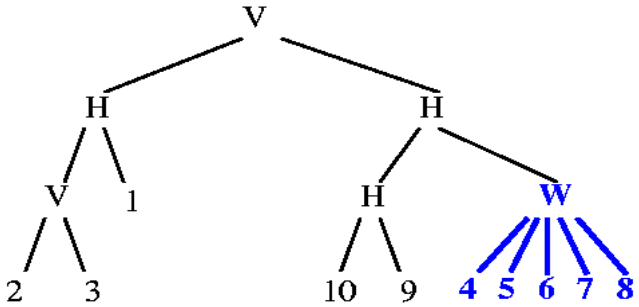
- **Wheel**: The smallest non-slicing floorplans (Wang & Wong, IEEE TCAD'92).
- **Floorplan of order 5.**
- **Floorplan tree**: A tree representing the hierarchy of partitioning.



The two possible wheels.



A floorplan of order 5



Corresponding floorplan tree

Slicing Floorplan Design by Simulated Annealing

- Related works
  - Wong & Liu, “A new algorithm for floorplan design,” DAC’86.
  - Wong, Leong & Liu, *Simulated Annealing of VLSI Design*, pp. 31-51, Kluwer Academic Publishers, 1988.
- Ingredients: solution space, neighborhood structure, cost function, annealing schedule.

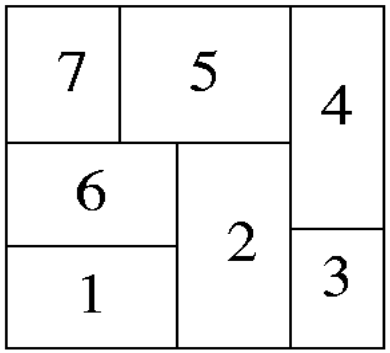
Solution Representation

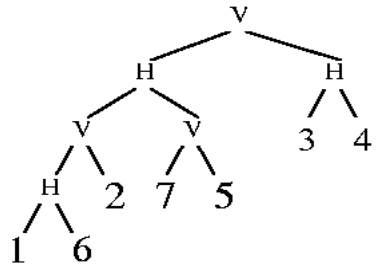
- An expression $E = e_1 e_2 \ldots e_{2n-1}$, where $e_i \in \{1, 2, \ldots, n, H, V\}$, $1 \le i \le 2n-1$, is a **Polish expression** of length $2n$-1 iff
  1. every operand j, $1 \le j \le n$, appears exactly once in E;
  2. (**balloting property**) for every sub-expression $E_i = e_1 \ldots e_i$, $1 \le i \le 2n-1$, #operands > #operators.

1 6 H 3 5 V 2 H V 7 4 H V

# of operands = 4 ..... = 7
# of operators = 2 ..... = 5

- Polish expression ↔ Postorder traversal.
- *ijH*: *i* below *j*; *ijV*: *i* on the left of *j*.



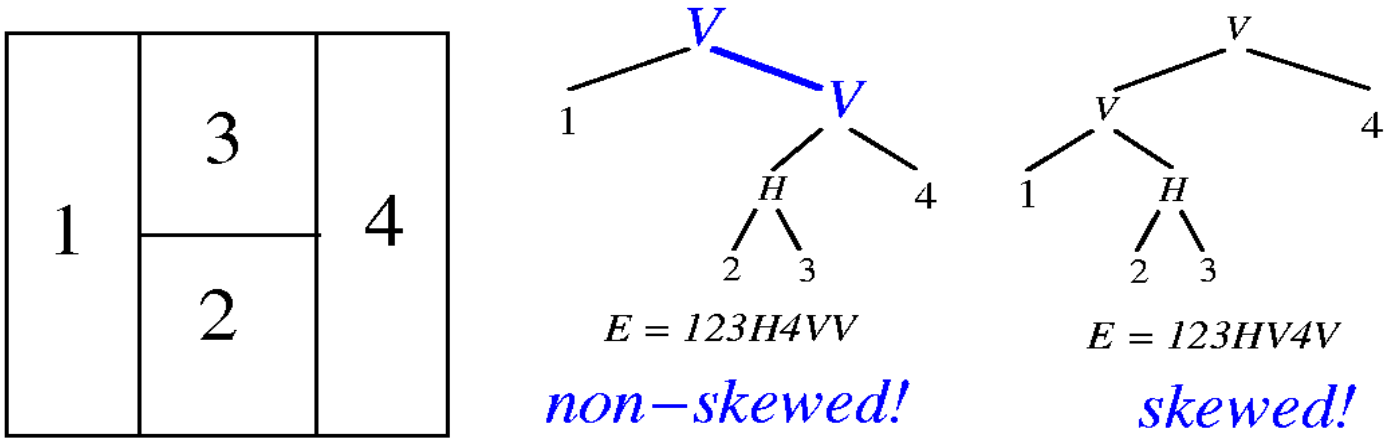


E = 16H2V75VH34HV

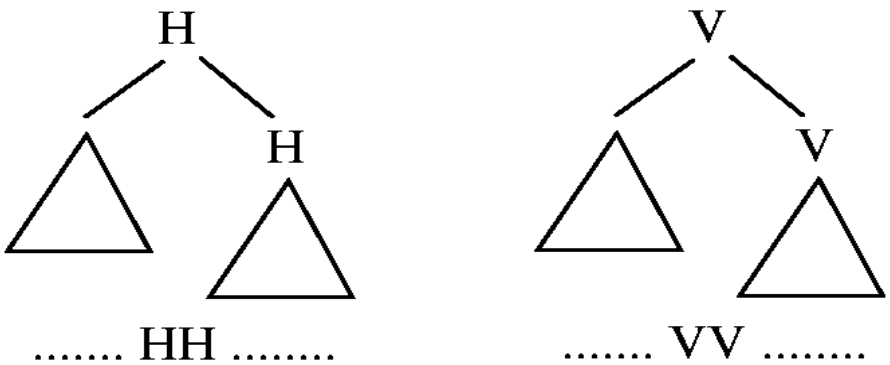
*E = 16+2*75*+34+**

Postorder traversal of a tree!

Solution Representation (cont'd)



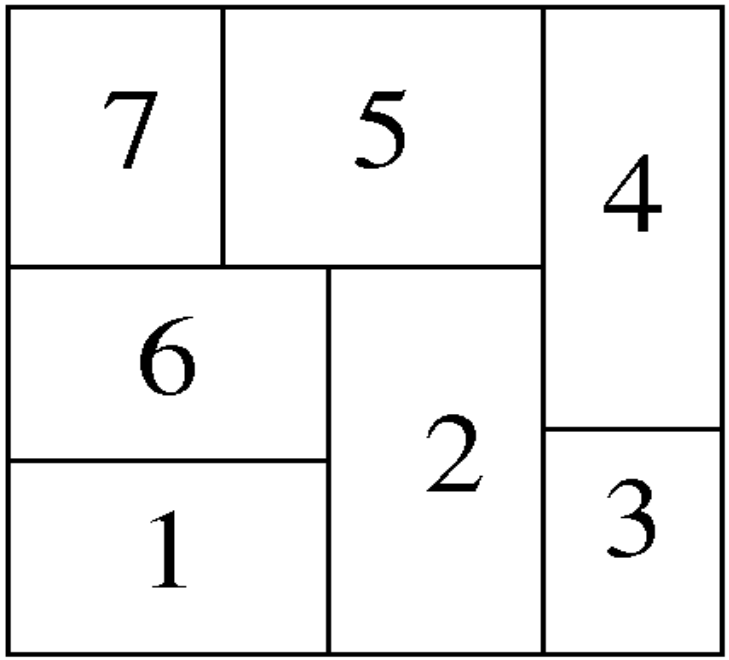
Non−skewed cases

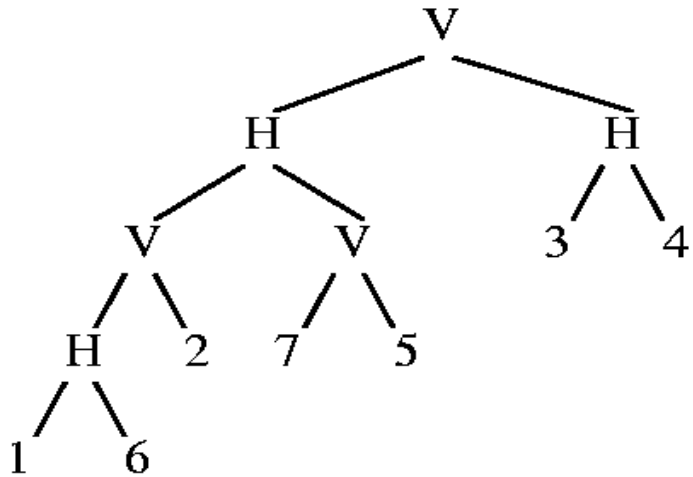


- **Question**: how to eliminate redundant representations?

Normalized Polish Expression

- A Polish expression $E = e_1 e_2 \ldots e_{2n-1}$ is called **normalized** iff E has no consecutive operators of the same type (H or V).
- Given a **normalized** Polish expression, we can construct a **unique** rectangular slicing structure.



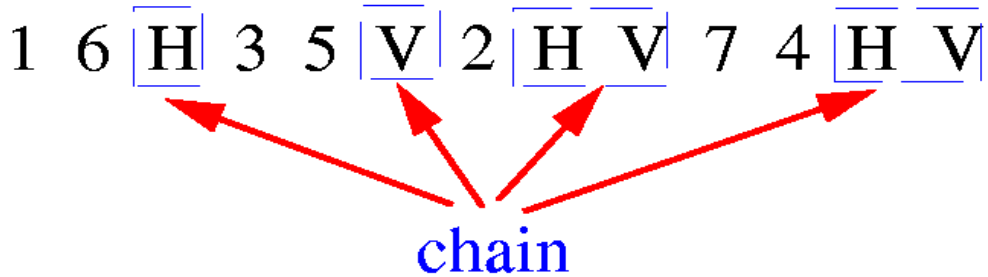


$E = 16H2V75VH34HV$

A normalized Polish expression

Neighborhood Structure

- **Chain**: *HVHVH*…or *VHVHV*…



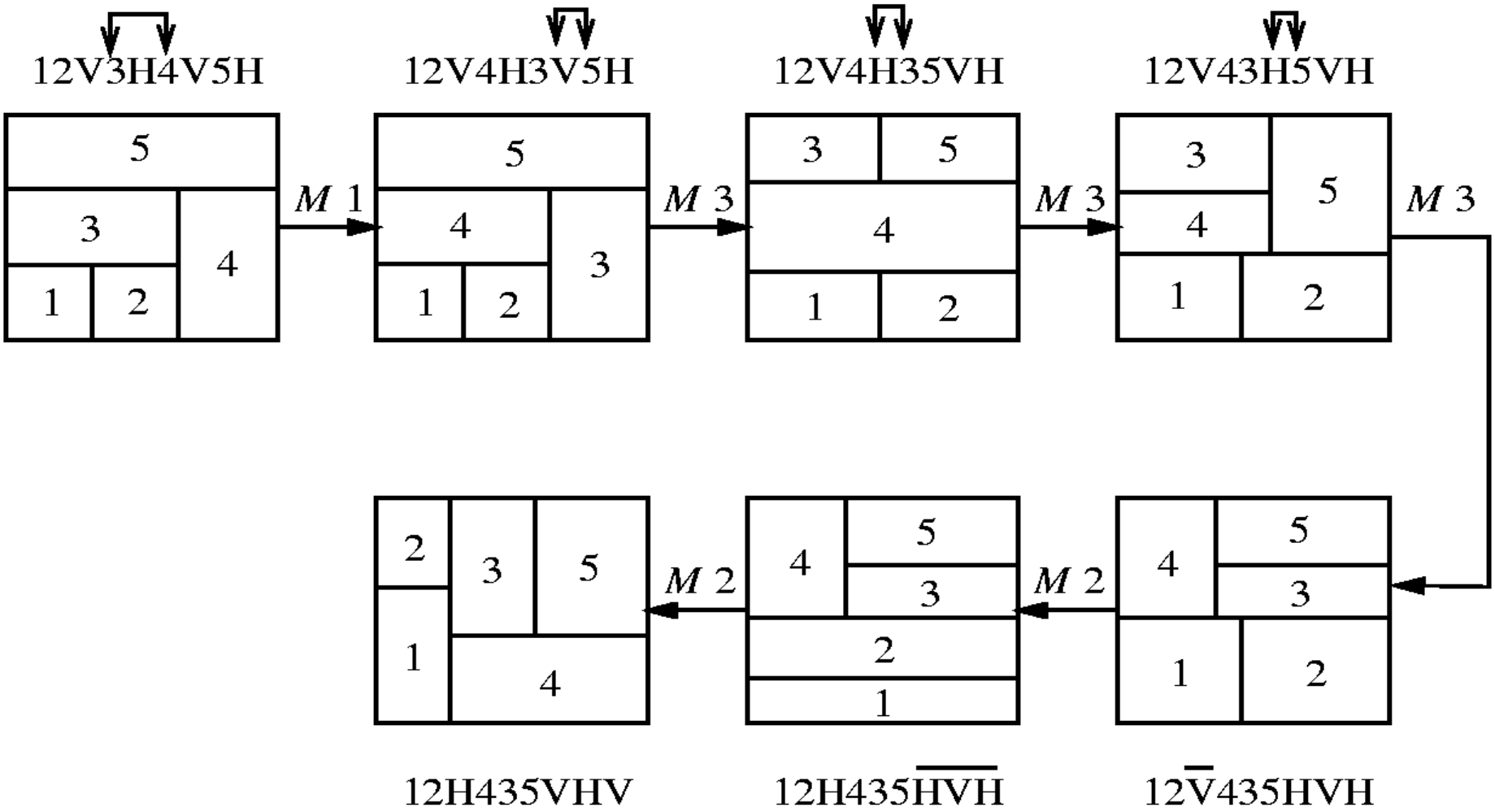
- **Adjacent**: 1 and 6 are adjacent operands; 2 and 7 are adjacent operands; 5 and *V* are adjacent operand and operator.
- 3 types of moves:

  - M_1 (**Operand Swap**): swap two adjacent operands.

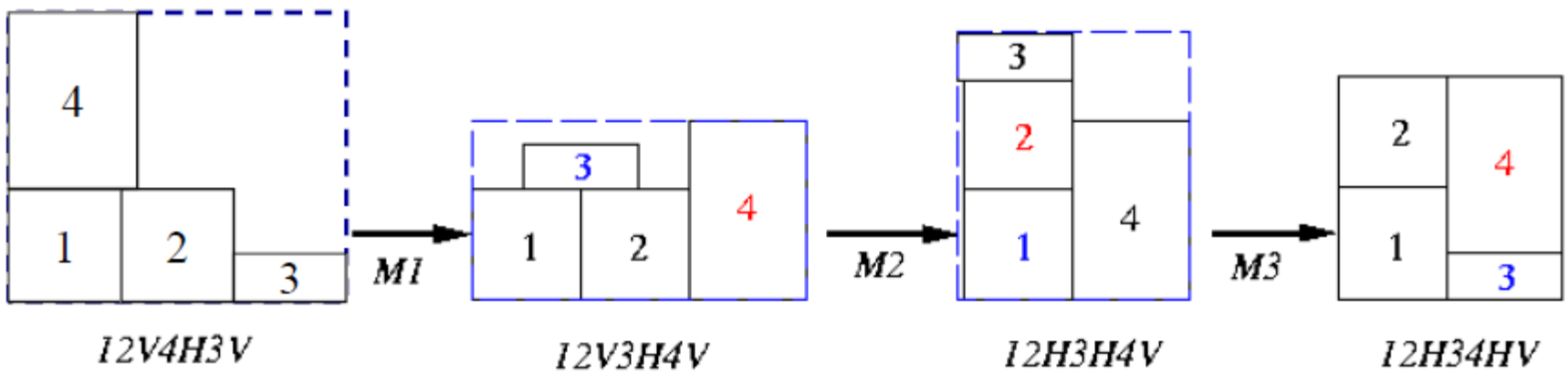
  - M_2 (**Chain Invert**): Complement a chain. $(\overline{V} = H, \overline{H} = V)$

  - M_3 (**Operator/Operand Swap**): Swap two adjacent operand and operator.
- It can be proved that each normalized Polish expression can be obtained from any other one through a finite set of moves of the above three types.

Solution Perturbation



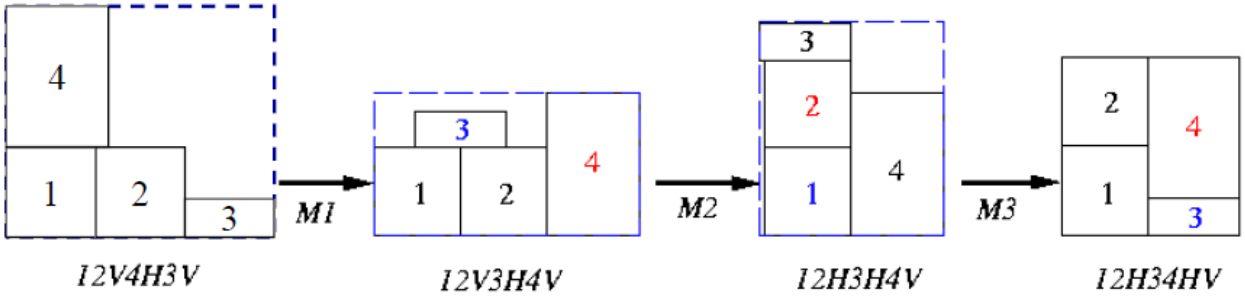
Effects of Perturbation



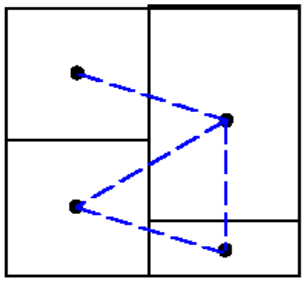
- **Question**: Does the balloting property hold during the moves?
  - M_1 and M_2 moves are OK.
  - Check the M_3 moves! Reject "illegal" M_3 moves.
- **Check M_3 moves**: Assume that the M_3 move swaps the operand e_i with the operator e_{i+1}, $1 \le i \le 2n-2$. Then, the swap will not violate the balloting property iff $2N_{i+1} < i$.
  - N_k: # of operators in the Polish expression $E = e_1 e_2 \ldots e_k$, $1 \le k \le 2n-1$.

Cost Function

- $\Phi = A + \lambda W$
  - A: area of the smallest rectangle
  - W: overall wiring length
  - λ: user-specified parameter

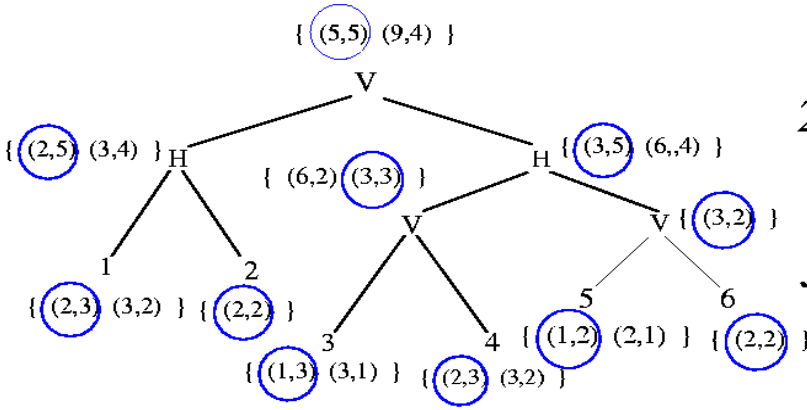


- $W = \sum_{ij} c_{ij} d_{ij}$
  - c_{ij}: # of connections between blocks i and j.
  - d_{ij}: center-to-center distance between basic rectangles i and j.

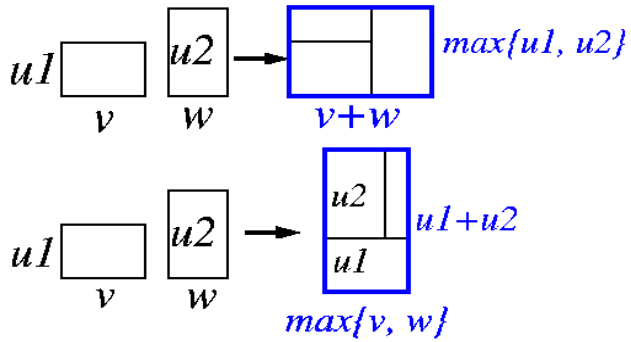


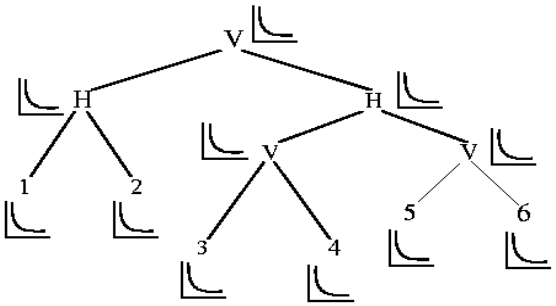
Area Computation for Hard Blocks

- Stockmeyer, "Optimal orientations of cells in slicing floorplan designs," *Information and Control*, 1983.
- Time complexity: $O(knd)$, where n is # modules, d is the depth of tree, and each module has $O(k)$ possible shapes.



	2	1	2
2	2	5	6
3	1	3	4



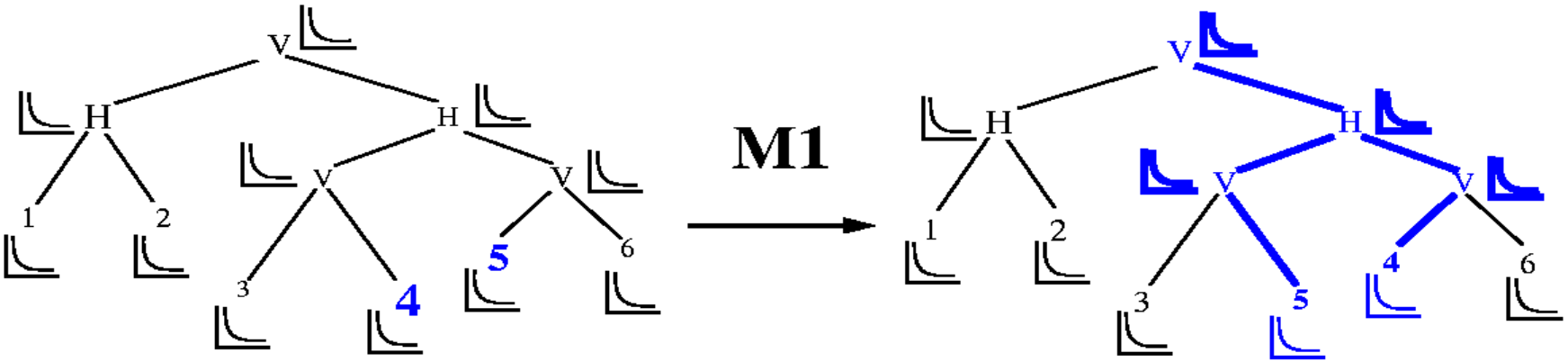


Slicing Floorplan Sizing

- The shape function of each leaf block is given as a staircase (or piecewise linear) function.
- Traverse the slicing tree to compute the shape functions of all composite blocks (bottom-up composition).
- Choose the desired shape of the top-level block
  - Only the corner points of the function need to be evaluated for area minimization.
- Propagate the consequences of the choice down to the leaf blocks (top-down propagation).

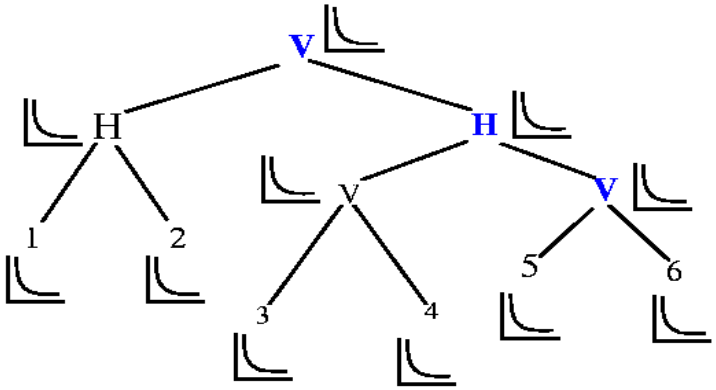
Incremental Area Computation

- Each move leads to only a minor modification of the Polish expression.
- At most **two paths** of the slicing tree need to be updated for each move.



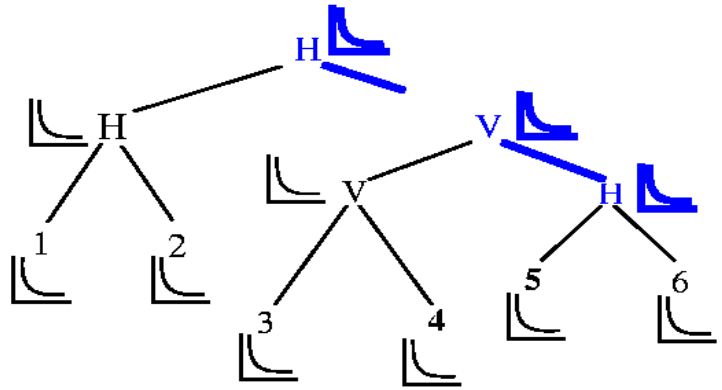
E = 12H34V56VHV **E = 12H35V46VHV**

Incremental Area Computation (cont'd)

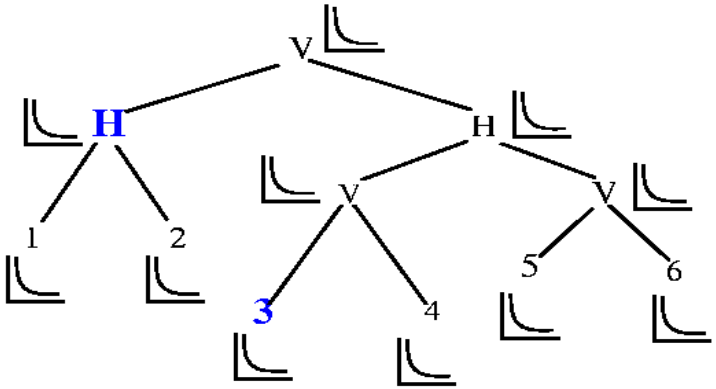


E = 12H34V56VHV

M2

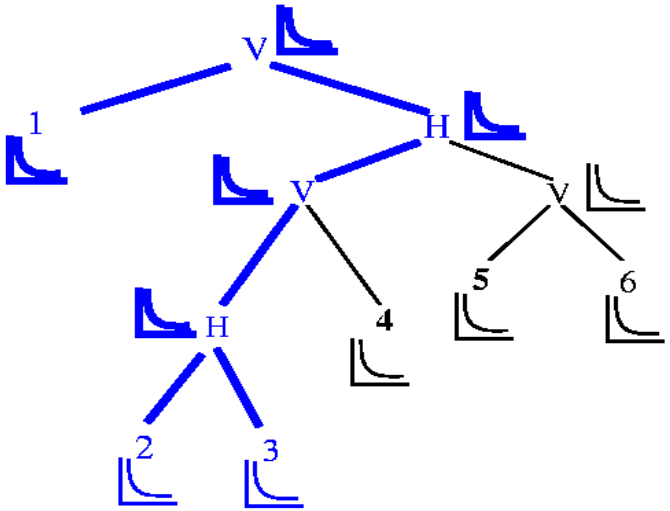


E = 12H34V56HVH



E = 12H34V56VHV

M3



E = 123H4V56VHV

Annealing Schedule

- Initial solution: $12V3V\ldots nV$

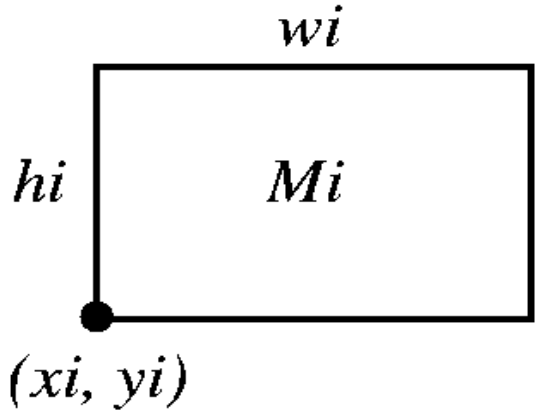
1	2	3		n

- $T_i = r^i T_0$, $i = 1, 2, 3, \ldots$; $r = 0.85$
- At each temperature, try kn moves (k = 5-10)
- Terminate the annealing process if
  - # of accepted moves < 5%
  - Temperature is low enough, or
  - Run out of time.

```
Algorithm: Simulated_Annealing_Floorplanning(P, ε, r, k)
begin
E ← 12V3V4V...nV; /* initial solution */
Best ← E; T_0 ← Δ_avg / ln(P); M ← MT ← uphill ← 0; N = kn;
repeat
    MT ← uphill ← reject ← 0;
    repeat
        SelectMove(M);
        Case M of
        M_1: Select two adjacent operands e_i and e_j; NE ← Swap(E, e_i, e_j);
        M_2: Select a nonzero length chain C; NE ← Complement(E, C);
        M_3: done ← FALSE;
            while not (done) do
                Select two adjacent operand e_i and operator e_{i+1};
                if (e_{i-1} ≠ e_{i+1}) and (2N_{i+1} < i) then done ← TRUE;
            NE ← Swap(E, e_i, e_{i+1});
        MT ← MT + 1; Δcost ← cost(NE) − cost(E);
        if (Δcost ≤ 0) or (Random < e^{−Δcost/T})
        then
            if (Δcost > 0) then uphill ← uphill + 1;
            E ← NE;
            if cost(E) < cost(best) then best ← E;
        else reject ← reject + 1;
    until (uphill > N) or (MT > 2N);
    T = rT; /* reduce temperature */
until (reject/MT > 0.95) or (T < ε) or OutOfTime;
end
```

Non-slicing Floorplan Design by Mathematical Programming

- Sutanthavibul, Shragowitz, and Rosen, "An analytical approach to floorplan design and optimization," DAC'90.
- Notation:
  - w_i, h_i: width and height of module M_i.
  - (x_i, y_i): coordinates of the lower left corner of module M_i.
  - $a_i \le w_i/h_i \le b_i$: aspect ratio w_i/h_i of module M_i. (Note: We defined aspect ratio as h_i/w_i before.)
- Goal: Find a **mixed integer linear programming (MILP)** formulation for the floorplan design.
  - **Linear** constraints? Objective function?



$$Area = hi * wi$$
$$Aspect\ ratio = wi / hi$$

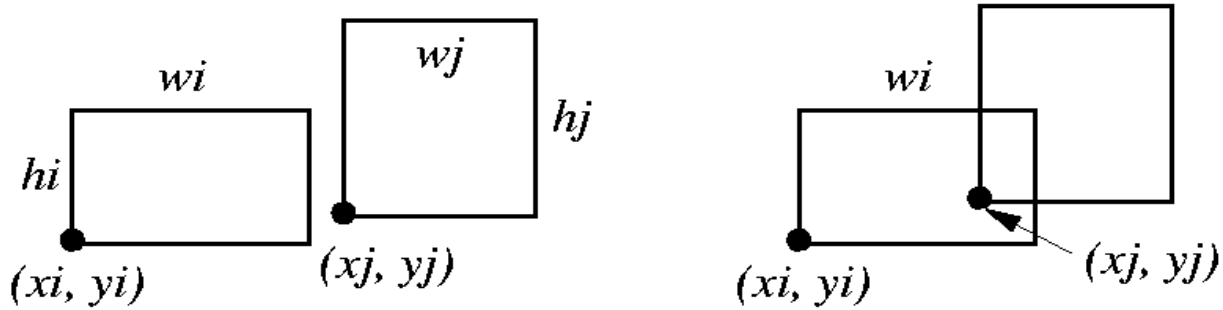
Non-overlap Constraints

- Two modules M_i and M_j do not overlap, if at least one of the following linear constraints is satisfied (cases encoded by p_{ij} and q_{ij}):

		p_{ij}	q_{ij}
M_i to the left of M_j:	$x_i + w_i \leq x_j$	0	0
M_i below M_j:	$y_i + h_i \leq y_j$	0	1
M_i to the right of M_j:	$x_i - w_j \geq x_j$	1	0
M_i above M_j:	$y_i - h_j \geq y_j$	1	1

- Let W, H be upper bounds on the floorplan width and height, respectively.
- Introduce two 0, 1 variables p_{ij} and q_{ij} to denote that one of the above inequalities is enforced; e. g., $p_{ij} = 0$, $q_{ij} = 1 \Rightarrow y_i + h_i \leq y_j$ is satisfied.

$$
\begin{aligned}
x_i + w_i &\leq x_j + W(p_{ij} + q_{ij}) \\
y_i + h_i &\leq y_j + H(1 + p_{ij} - q_{ij}) \\
x_i - w_j &\geq x_j - W(1 - p_{ij} + q_{ij}) \\
y_i - h_j &\geq y_j - H(2 - p_{ij} - q_{ij})
\end{aligned}
$$



$xi + wi <= xj$ $\qquad$ $xi + wi > xj$

Cost Function & Constraints

- Minimize Area = xy, nonlinear! (x, y: width and height of the resulting floorplan)
- How to fix?

  - Fix the width W and minimize the height y!
- Four types of constraints:
  1. No two modules overlap ($\forall i, j$: $1 \le i < j \le n$);
  2. Each module is enclosed within a rectangle of width W and height y ($x_i + w_i \le W$, $y_i + h_i \le y$, $1 \le i \le n$);
  3. $x_i \ge 0$, $y_i \ge 0$, $1 \le i \le n$;
  4. $p_{ij}, q_{ij} \in \{0, 1\}$.
- w_i, h_i are known.

Mixed ILP for Floorplanning

Mixed ILP for the floorplanning problem with rigid, fixed modules.

$$\min \quad y$$

subject to

$$\begin{aligned}
x_i + w_i &\le W, & 1 \le i \le n \quad &(1)\\
y_i + h_i &\le y, & 1 \le i \le n \quad &(2)\\
x_i + w_i &\le x_j + W(p_{ij} + q_{ij}), & 1 \le i < j \le n \quad &(3)\\
y_i + h_i &\le y_j + H(1 + p_{ij} - q_{ij}), & 1 \le i < j \le n \quad &(4)\\
x_i - w_j &\ge x_j - W(1 - p_{ij} + q_{ij}), & 1 \le i < j \le n \quad &(5)\\
y_i - h_j &\ge y_j - H(2 - p_{ij} - q_{ij}), & 1 \le i < j \le n \quad &(6)\\
x_i, y_i &\ge 0, & 1 \le i \le n \quad &(7)\\
p_{ij}, q_{ij} &\in \{0, 1\}, & 1 \le i < j \le n \quad &(8)
\end{aligned}$$

- Size of the mixed ILP: for n modules,
  - # continuous variables: $O(n)$; # integer variable: $O(n^2)$; # linear constraints: $O(n^2)$.
  - Unacceptably huge program for a large n!
- Popular LP software: LINDO, lp_solve, CPLEX, GUROBI, etc.

Mixed ILP for Floorplanning (cont'd)

$$\min \quad y$$

subject to

$$x_i + r_i h_i + (1 - r_i) w_i \le W, \qquad 1 \le i \le n \qquad (9)$$

$$y_i + r_i w_i + (1 - r_i) h_i \le y, \qquad 1 \le i \le n \qquad (10)$$

$$x_i + r_i h_i + (1 - r_i) w_i \le x_j + M(p_{ij} + q_{ij}), \qquad 1 \le i < j \le n \qquad (11)$$

$$y_i + r_i w_i + (1 - r_i) h_i \le y_j + M(1 + p_{ij} - q_{ij}), \qquad 1 \le i < j \le n \qquad (12)$$

$$x_i - r_j h_j - (1 - r_j) w_j \ge x_j - M(1 - p_{ij} + q_{ij}), \qquad 1 \le i < j \le n \qquad (13)$$

$$y_i - r_j w_j - (1 - r_j) h_j \ge y_j - M(2 - p_{ij} - q_{ij}), \qquad 1 \le i < j \le n \qquad (14)$$

$$x_i, y_i \ge 0, \qquad 1 \le i \le n \qquad (15)$$

$$p_{ij}, q_{ij} \in \{0, 1\}, \qquad 1 \le i < j \le n \qquad (16)$$

- For each module *i* with free orientation, associate a 0-1 variable r_i;
- $r_i = 0$: 0° rotation for module *i*.
- $r_i = 1$: 90 ° rotation for module *i*.
- $M = \max\{W, H\}$.

Soft Modules

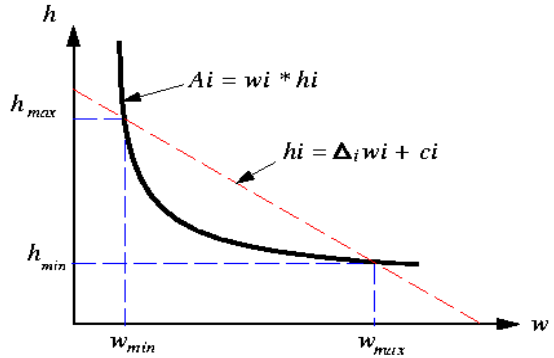
- Assumptions: w_i, h_i are unknown; area lower bound: A_i.
- Module size constraints: $w_i h_i \geq A_i$; $a_i \leq \frac{w_i}{h_i} \leq b_i$.
- Hence, $w_{min} = \sqrt{A_i a_i}$, $w_{max} = \sqrt{A_i b_i}$, $h_{min} = \sqrt{\frac{A_i}{b_i}}$, $h_{max} = \sqrt{\frac{A_i}{a_i}}$
- $w_i h_i \geq A_i$ nonlinear! How to fix? (The following fixing scheme is different from the one given in the paper.)
  - Can apply a first-order approximation of the equation: a line passing through (w_{min}, h_{max}) and (w_{max}, h_{min}).

$$h_i = \Delta_i w_i + c_i \qquad /* \; y = mx + c \; */$$

$$\Delta_i = \frac{h_{max} - h_{min}}{w_{min} - w_{max}} \qquad /* \; slope \; */$$

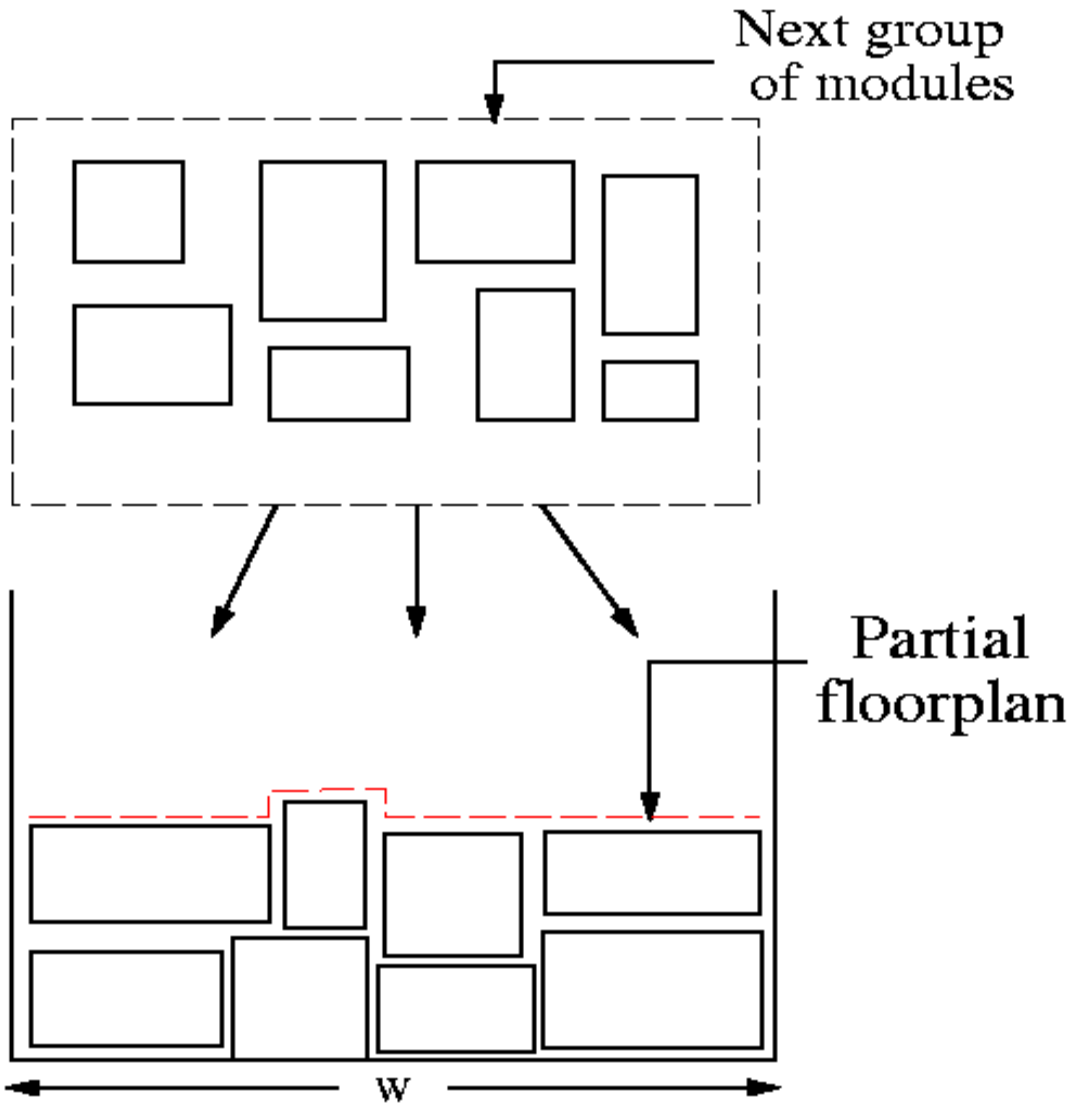
$$c_i = h_{max} - \Delta_i w_{min} \qquad /* \; c = y_0 - mx_0 \; */$$

  - Substitute $\Delta_i w_i + c_i$ for h_i to form linear constraints (x_i, y_i, w_i are unknown; Δ_i, c_i, h_i can be computed as above).



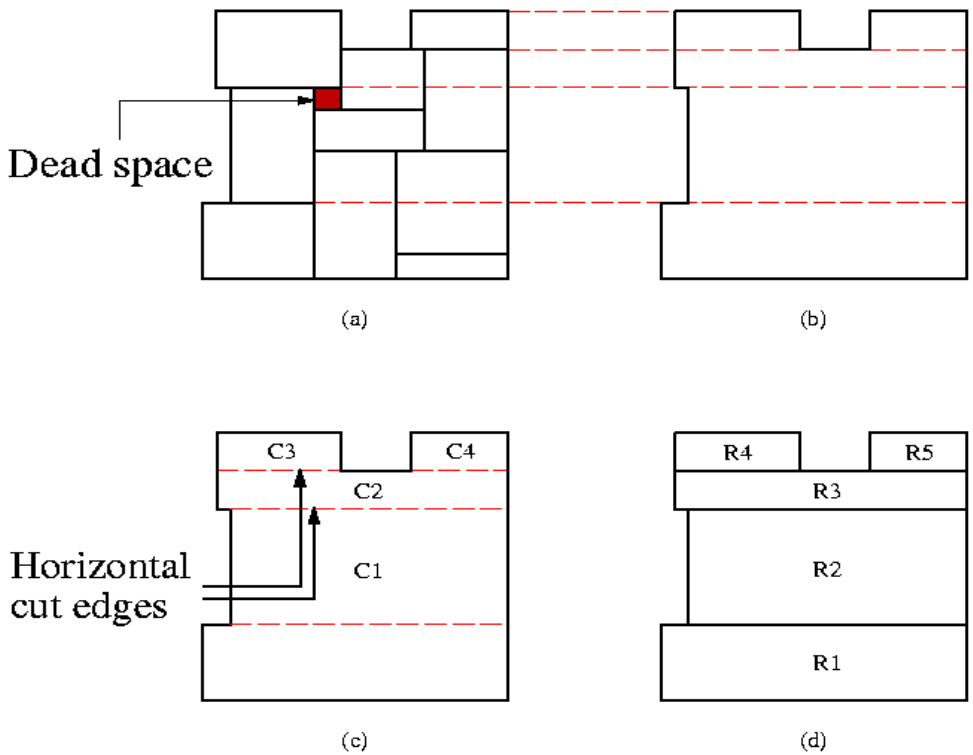
Reducing the Size of the Mixed ILP

- Time complexity of a mixed ILP: exponential!
- Recall the large size of the mixed ILP: # variables, # constraints: $O(n^2)$.
  - How to fix it?
- Key: solve a partial problem at each step (successive augmentation)
- Questions:
  - How to select next subgroup of modules? $\Rightarrow$linear ordering based on connectivity.
  - How to minimize # of required variables?


Next group
of modules
Partial
floorplan
w

Reducing the Size of the Mixed ILP (cont'd)

- Size of each successive mixed ILP depends on (1) # of modules in the next group; (2) "size" of the partially constructed floorplan.
- Keys to deal with (2):
  - Minimize the problem size of the partial floorplan.
  - Replace the already placed modules by a set of covering rectangles.
  - # rectangles is usually much smaller than # placed modules.



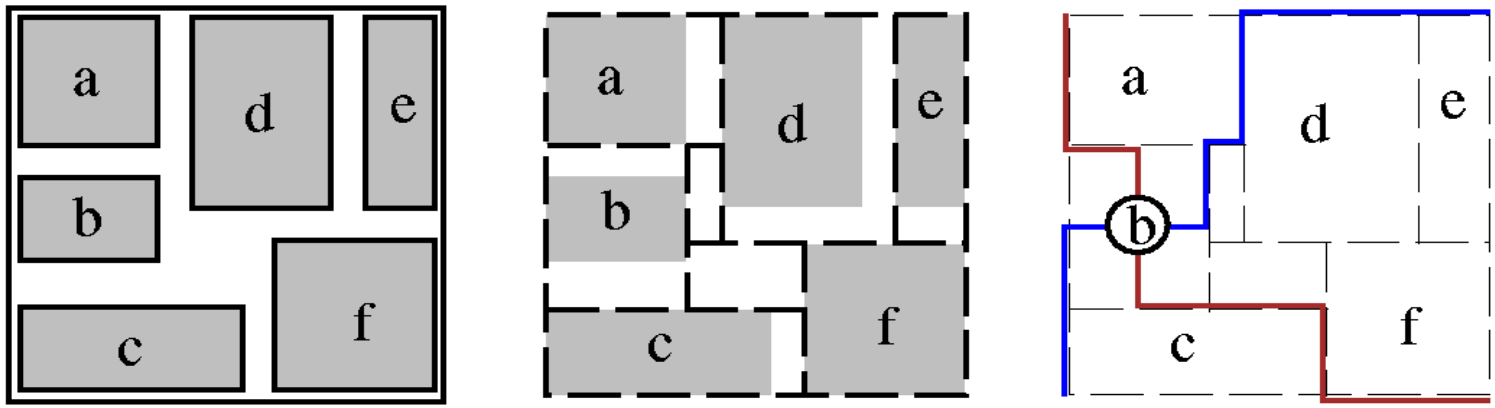
P-admissible Solution Space

- **P-admissible** solution space for Problem *P*:
  1. the solution space is finite,
  2. every solution is feasible,
  3. evaluation for each configuration is possible in polynomial time and so is the implementation of the corresponding configuration, **and**
  4. the configuration corresponding to the best evaluated solution in the space coincides with an optimal solution of *P*.
- Slicing floorplan is **not** P-admissible. Why?
- A P-admissible floorplan representation:

  Sequence-Pair.

Sequence Pair (SP)

- Murata, Fujiyoshi, Nakatake and Kajitani, "Rectangle-packing-based module placement," ICCAD'95.
- Represent a *packing* by a pair of module permutations called sequence-pair (e.g., (*abdecf*, *cbfade*)).
- The set of all sequence-pairs is a P-admissible solution space whose size is $(n!)^2$.
- Search in the P-admissible solution space by simulated annealing.
  - Swap two modules in the first sequence.
  - Swap two modules in both sequences.
  - Rotate a module.

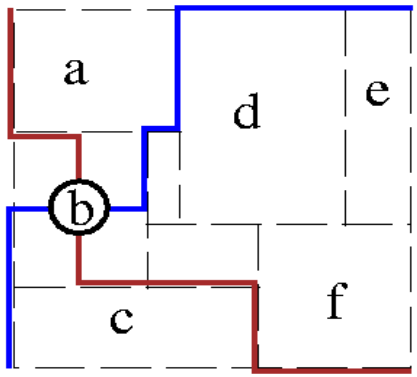


A floorplan

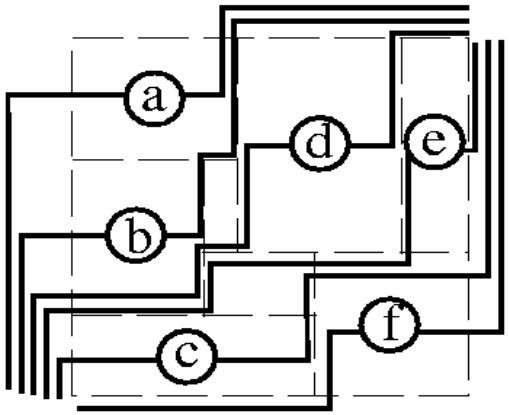
Loci of module b

Relative Module Positions

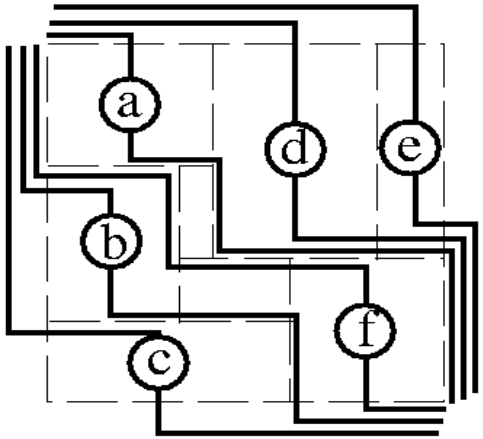
- A floorplan is a partition of a chip into rooms, each containing at most one module.
- **Locus** (right-up, left-down, up-left, down-right)
  1. Take a non-empty room.
  2. Start at the center of the room, walk in two alternating directions to hit the sides of rooms.
  3. Continue until to reach a corner of the chip.
- **Positive locus**: Union of right-up locus and left-down locus.
- **Negative locus**: union of up-left locus and down-right locus.



Loci of module b



Positive loci: abdecf

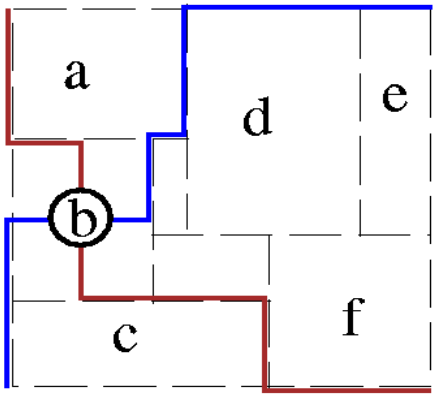


Negative loci: cbfade

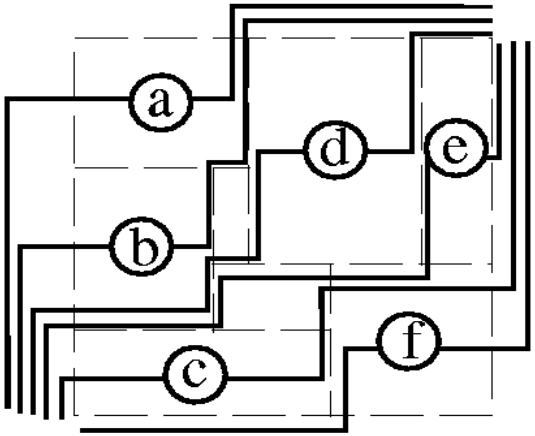
Geometrical Information

- No pair of positive (negative) loci cross each other, i.e., **loci are linearly ordered**.
- **Sequence-Pair** (Γ_+, Γ_-): Γ_+ (Γ_-) is the module permutation representing the order of positive (negative) loci.

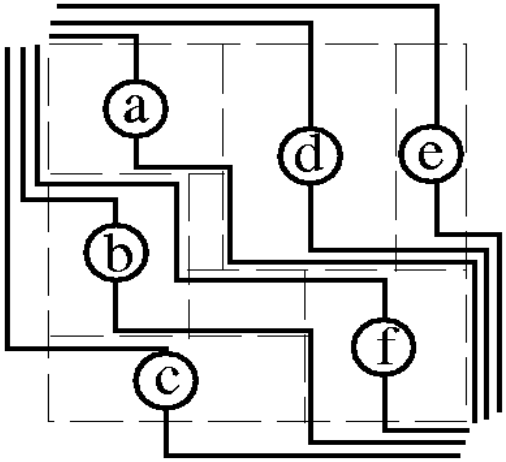
  E.g., (Γ_+, Γ_-) = (*abdecf*, *cbfade*).
- *x*' is **after** (**before**) *x* in both Γ_+ and Γ_- $\Rightarrow$ *x*' is **right** (**left**) to *x*.
- *x*' is **after** (**before**) *x* in Γ_+ and before (**after**) *x* in Γ_- $\Rightarrow$ *x*'is **below** (**above**) *x*.



Loci of module b



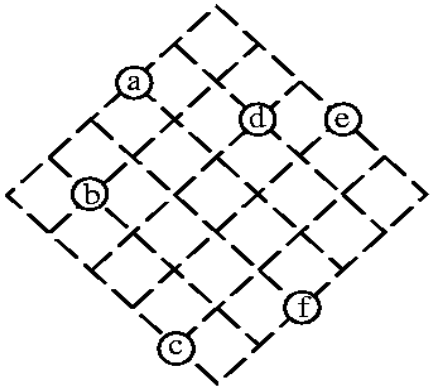
Positive loci: abdecf



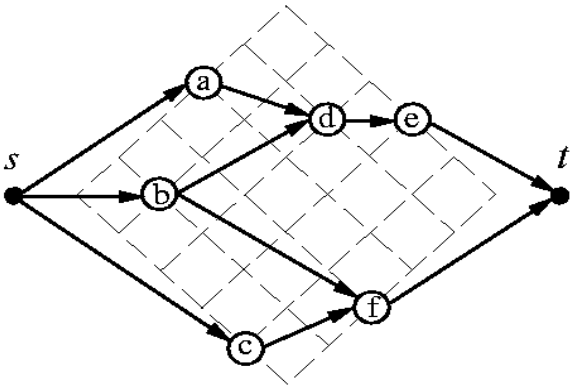
Negative loci: cbfade

Optimal (Γ_+, Γ_-)-Packing

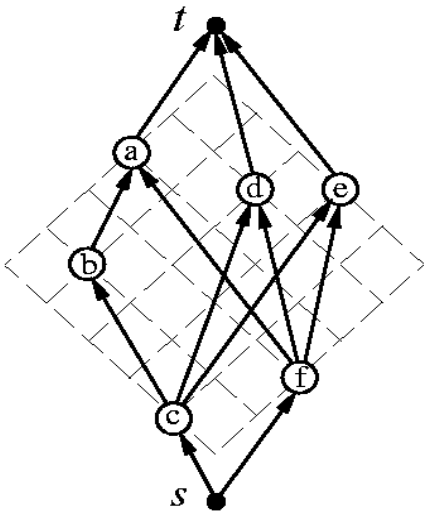
- For every sequence-pair (Γ_+, Γ_-), there is an optimal (Γ_+, Γ_-)-packing.
- **Horizontal constraint graph** $G_H(V, E)$ (similarly for $G_V(V, E)$):
  - V: source s, sink t, n vertices for modules.
  - E: (s, x) and (x, t) for each module x, and (x, x') iff x must be left-to x'.
  - **Vertex weight**: 0 for s and t, **width** of module x for the other vertices.



Packing for sequence pair: (abdecf, cbfade)



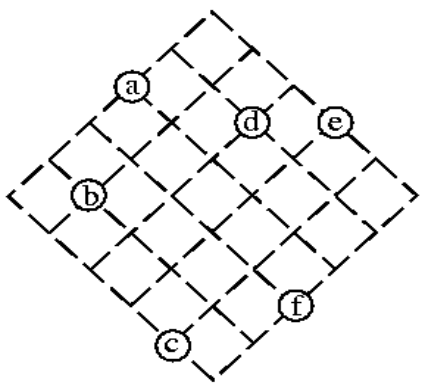
Horizontal constraint graph (Transitive edges are not shown)



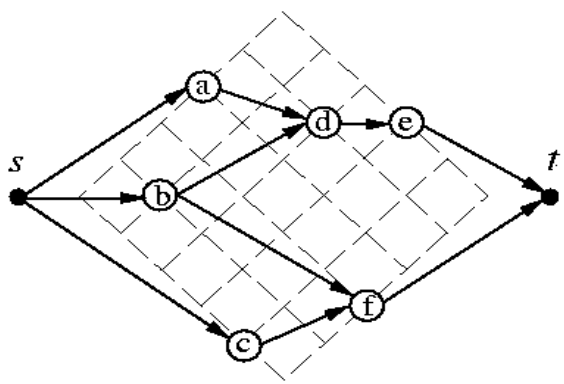
Vertical constraint graph (Transitive edges are not shown)

Optimal (Γ_+, Γ_-)-Packing (cont'd)

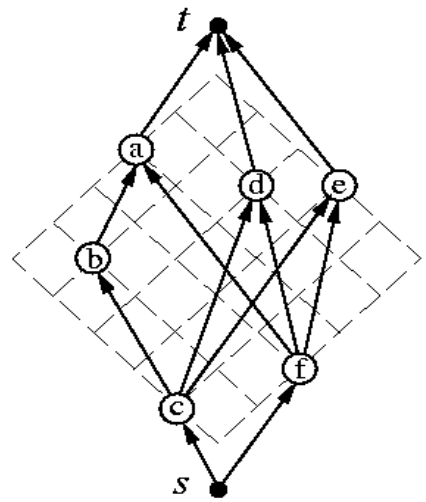
- **Optimal (Γ_+, Γ_-)-packing** can be obtained in $O(n^2)$ time by applying a longest path algorithm on a vertex-weighted directed acyclic graph.
  - G_H and G_V are independent.
  - The x and y coordinates of each module are determined by assigning the longest path length between s and the vertex of the module in G_H and G_V, respectively.
- More efficient algorithms for obtaining optimal (Γ_+, Γ_-)-packing: $O(n\log n)$ by Takahashi, IEICE'96; $O(n\log n)$ by Tang, Tian & Wong, DATE'00; $O(n\log\log n)$ by Tang & Wong, ASP-DAC'01.



Packing for sequence pair: (abdecf, cbfade)



Horizontal constraint graph (Transitive edges are not shown)



Vertical constraint graph (Transitive edges are not shown)

O-Tree

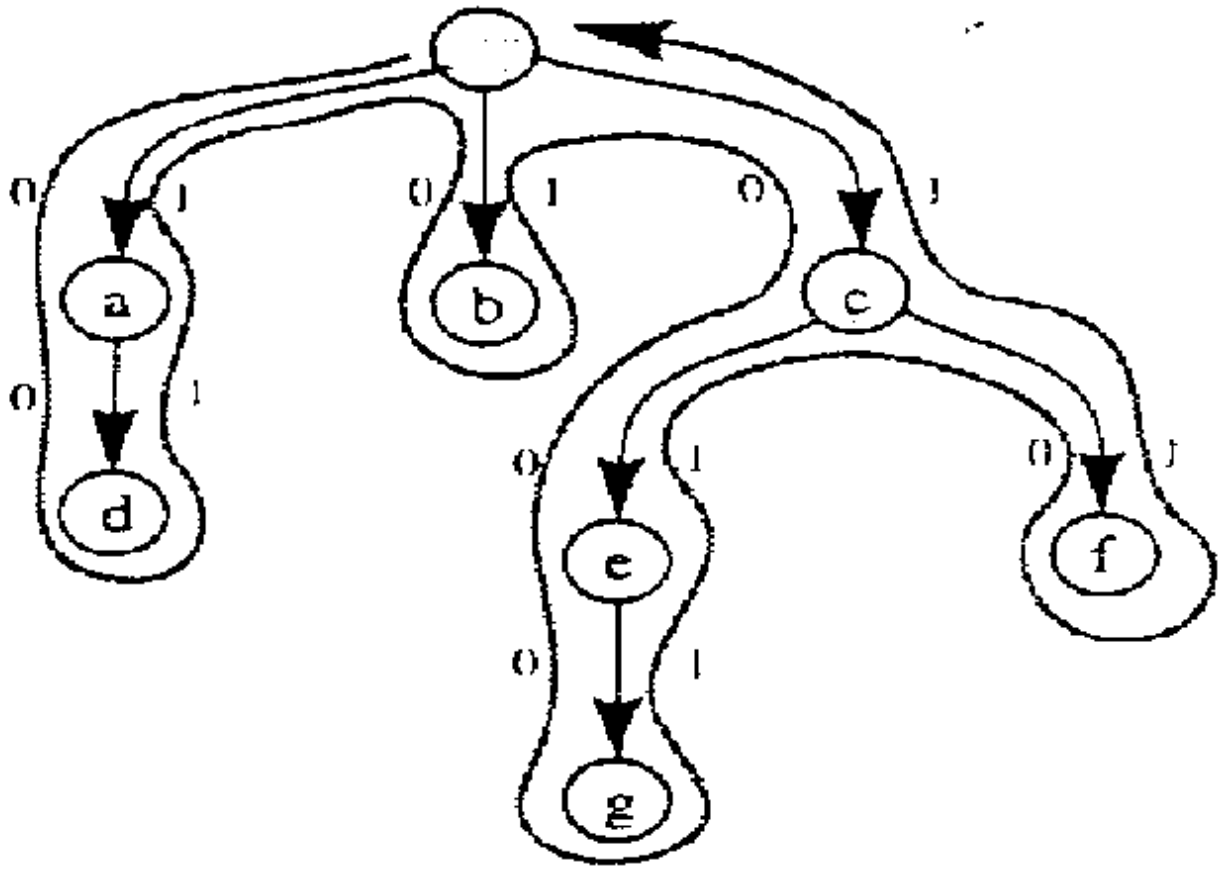
- Guo, Cheng, and Yoshimura, "An o-tree representation of non-slicing floorplan and its applications," DAC'99.
- Definitions:

  - A placement is **L-compact** (**B-compact**) if and only if no module can be moved left (down) from its original position with other modules' positions fixed.

  - A placement is **LB-compact** (or **admissible**) if and only if it is both L-compact and B-compact.
- Given any placement P_1, a corresponding admissible placement P_2 can be obtained by a sequence of x-direction and y-direction compactions. The overall area of P_2 is no larger than the overall area of P_1.

O-Tree Encoding

(00110100011011,*adbcegf*) with Depth-First-Search



O-Tree Encoding (cont'd)

- **Space needed to store (T,π):** Given a tree with n nodes in addition to its root, the label of each node can be encoded into a $\lceil \lg n \rceil$ bit string, and hence $n(2+\lceil \lg n \rceil)$ bits are needed to store (T,π) where $2n$ bits for T, and $n\lceil \lg n \rceil$ bits for π.

- **Number of possible (T,π)'s:** $O(n!2^{2n-2}/n^{1.5})$

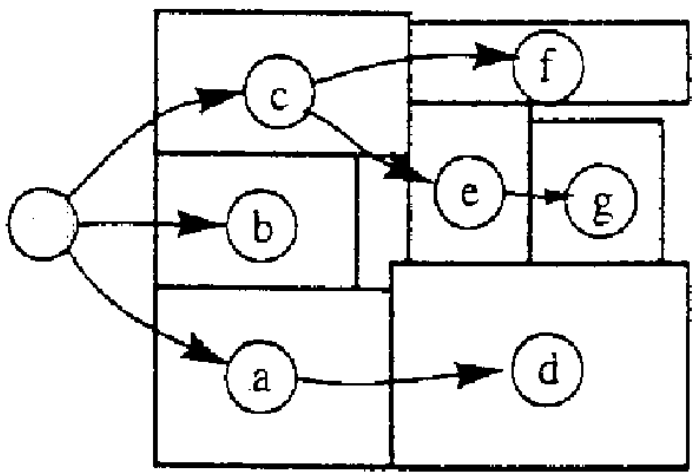
O-Tree and Placement

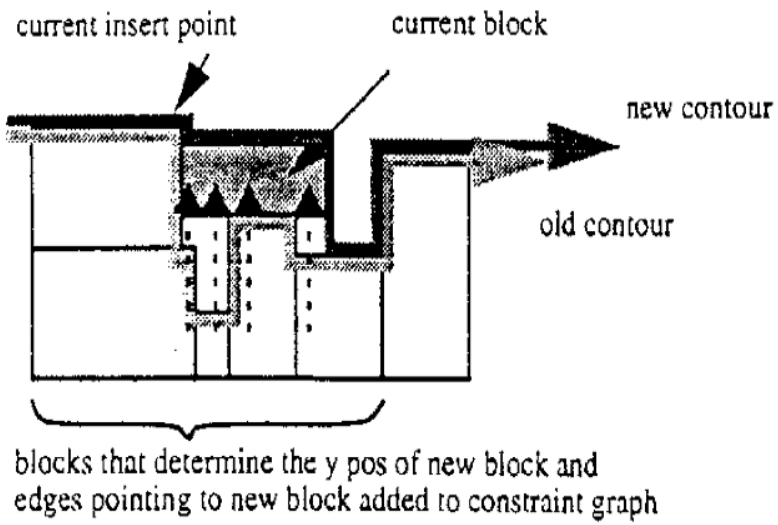
- **Horizontal O-Tree:**

  - Suppose i is the parent of j, $\Psi(j)$ is the set of blocks each of which appears before j in π and overlaps with j in the x-coordinate projections.

(0011010001101 1,*adbcegf*)

$$\begin{cases} x_j = x_i + w_i \\ y_j = 0 \mid (\max_{k \in \psi(j)} y_k + h_k) \end{cases}$$

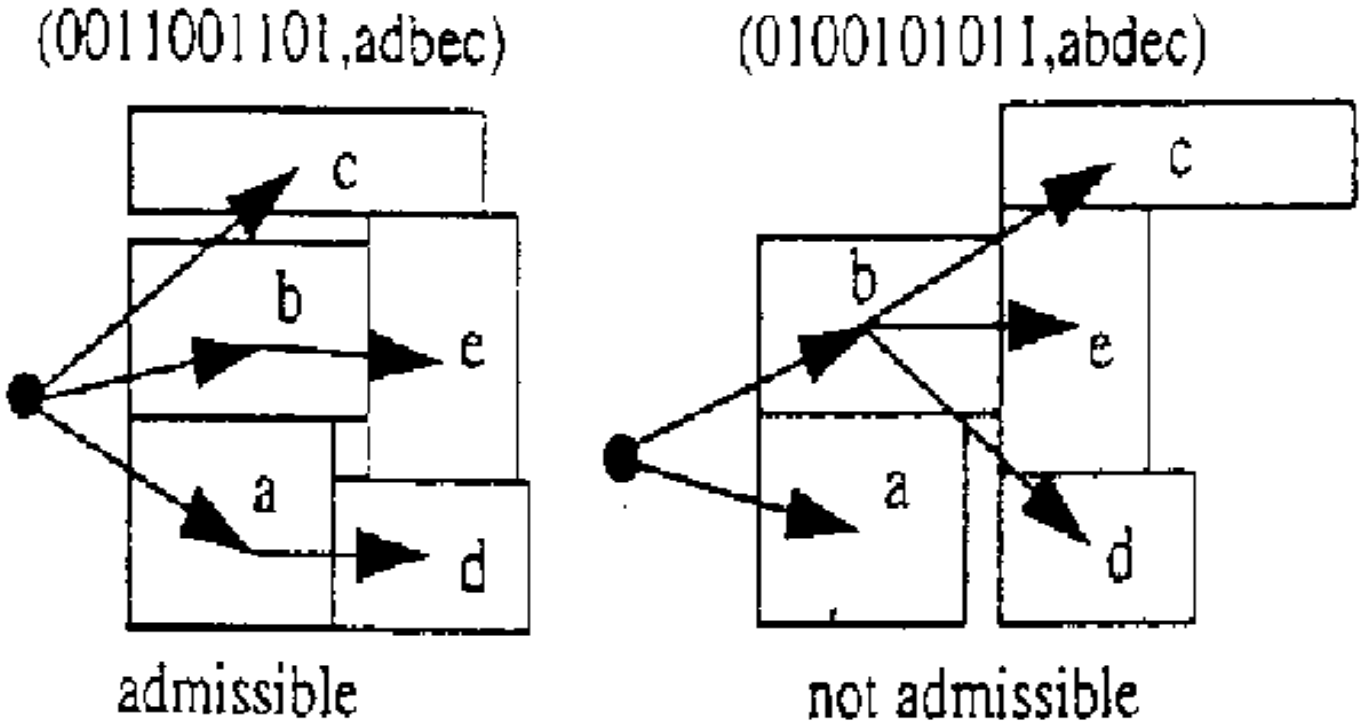




- **Vertical O-Tree:** can be defined similarly

O-Tree and Placement (cont'd)

An O-tree is **admissible** if its corresponding placement is admissible.



Admissible O-Tree Transformation (AOT)

- Given a horizontal O-tree T, we can first get a vertical constraint graph G_y by applying "OT2OCG" to T in linear time, and then get a vertical O-tree T_y by applying "CG2OT" to G_y in linear time. After applying the same procedures OT2OCG and CG2OT again, we can get another horizontal O-tree. The OT2OCG and CG2OT are iterated until an admissible O-tree is found.

$$\textbf{H-O-tree} \xrightarrow{\textbf{OT2OCG}} \textbf{G}_{\textbf{v}}\textbf{(B-compact)} \xrightarrow{\textbf{CG2OT}} \textbf{V-O-tree} \xrightarrow{\textbf{OT2OCG}} \textbf{G}_{\textbf{h}}\textbf{(L-compact)} \xrightarrow{\textbf{CG2OT}} \textbf{H-O-tree...}$$

- All compactions are **monotone** because modules are either moved down or left. Therefore, convergence of the above iteration is assured and we can get an admissible O-tree.

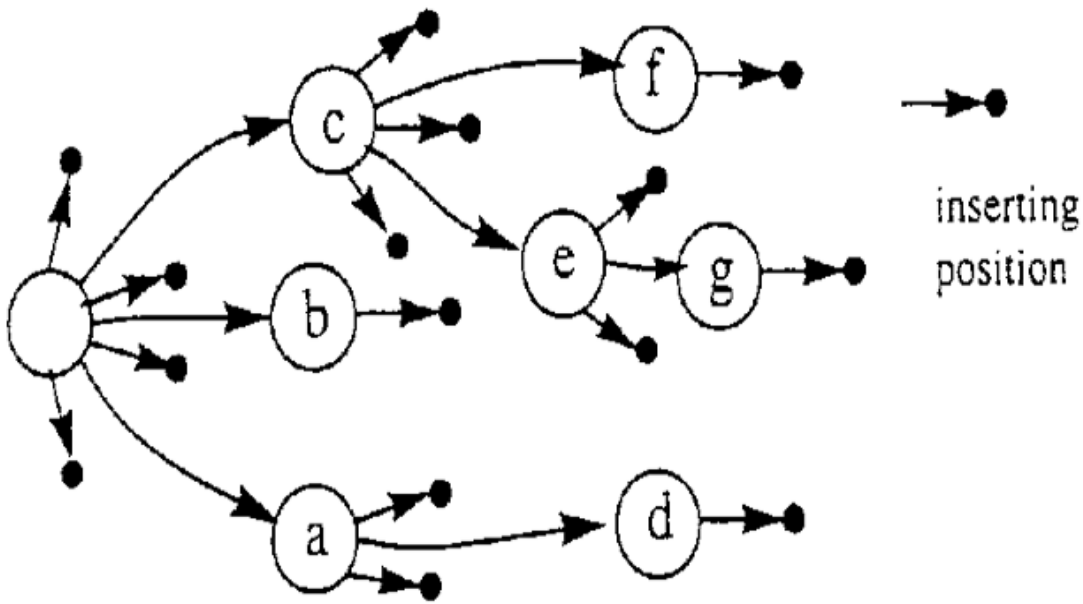
Solution Perturbation

a Select a module B_i in the O-tree (T,π).

b Delete B_i from the O-tree (T,π).

c Insert B_i in the position with the best cost value among all possible inserting positions in (T,π) as an external node.

d Perform a-c on it orthogonal O-tree.



•Given any O-tree with n nodes, the number of possible inserting position as external nodes is $2n$-1.

A Deterministic Placement Algorithm

- Perturb O-trees in sequence.

  - Select nodes in sequence and find the best perturb position for each of them.

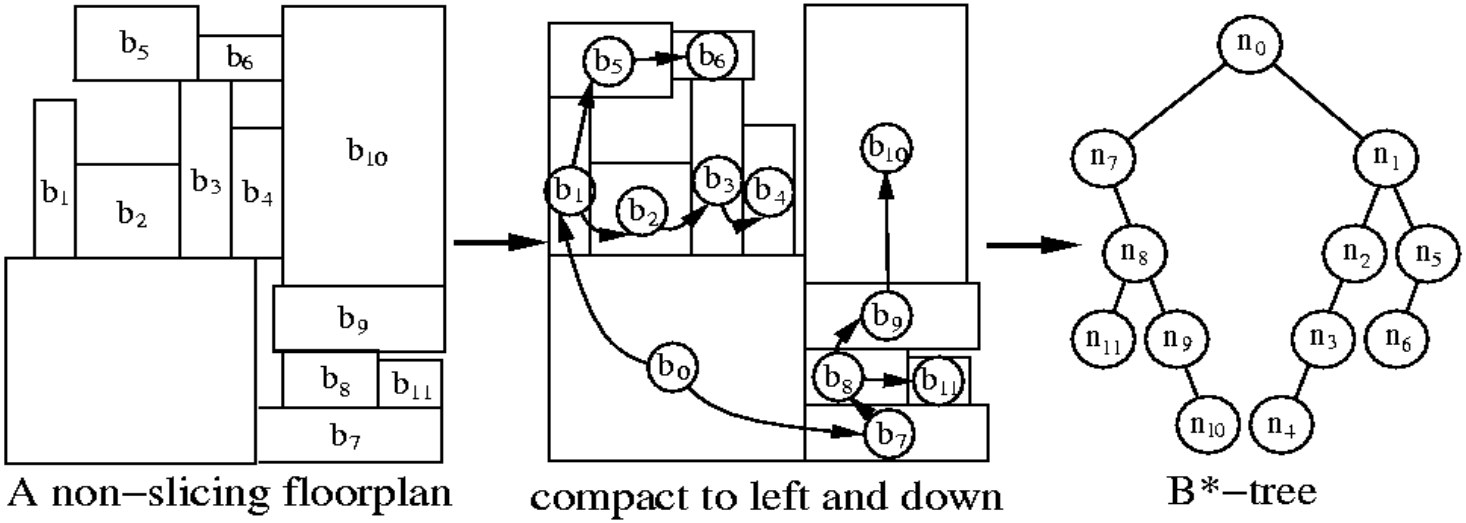
  - A perturbed O-tree can be made admissible using AOT.

- Implementation is straightforward.

B*-Tree

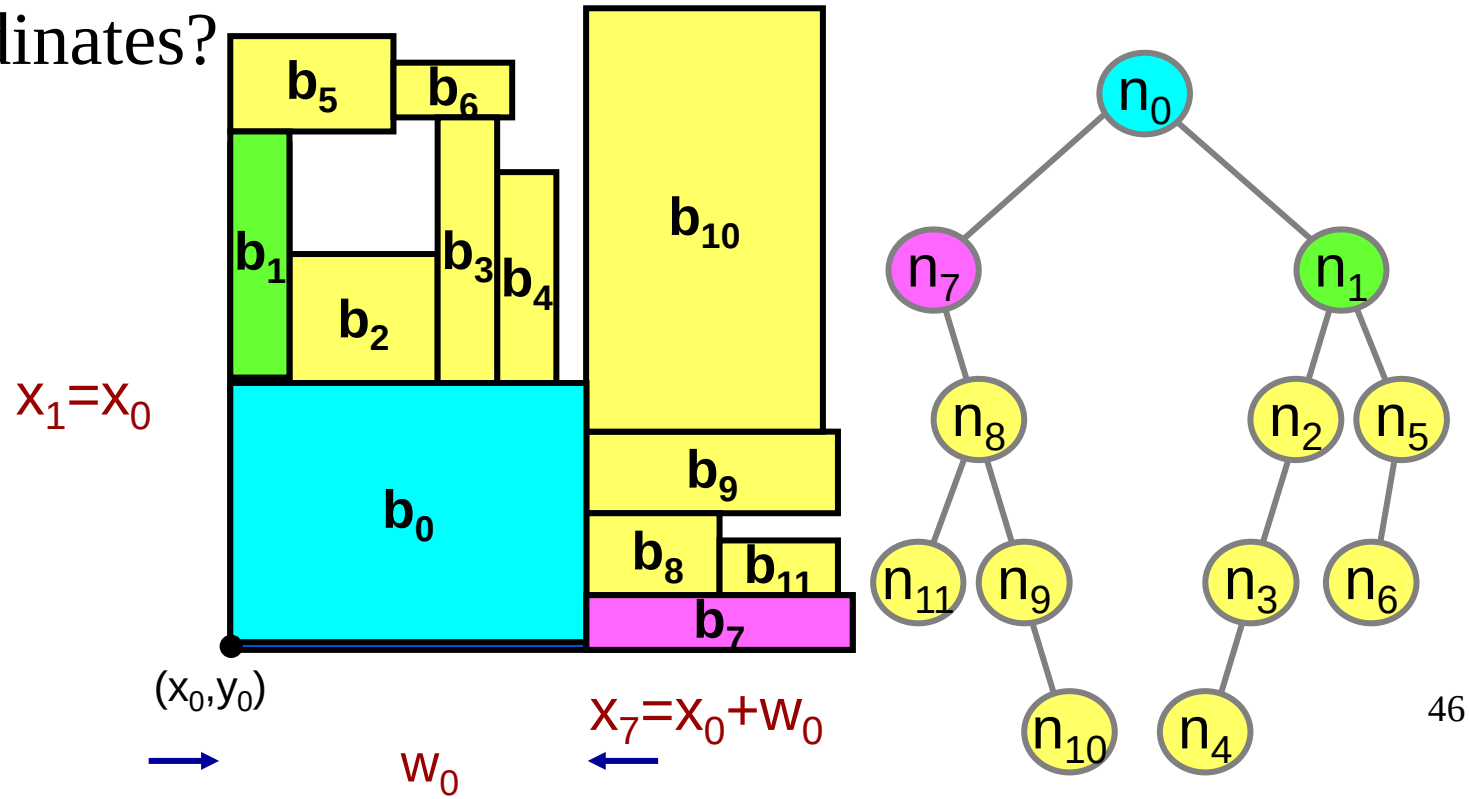
- Chang, Chang, Wu, and Wu, “B*-tree: a new representation for non-slicing floorplans,” DAC’00.
- Ideas:

  - From an admissible placement to a B*-tree:
    Left child: the lowest module on the right.
    Right child: the module above, with the same left-side coordinate.



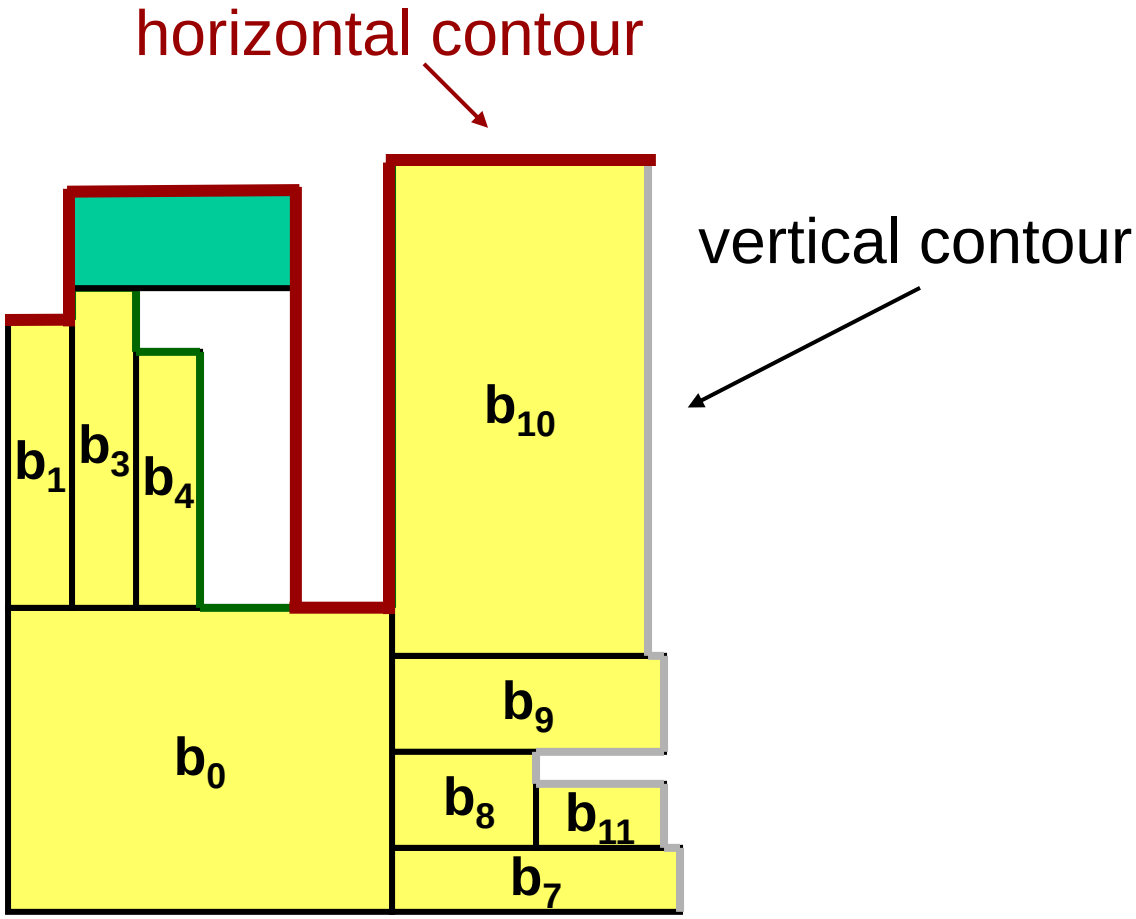
B*-tree Packing

- x-coordinates can be determined by the tree structure.
  - Left child: the lowest, adjacent block on the right ($x_j = x_i + w_i$).
  - Right child: the first block above, with the same *x*-coordinate ($x_j = x_i$).
- y-coordinates?



Computing y-coordinates

- Reduce the complexity of computing a y-coordinate to amortized O(1) time.

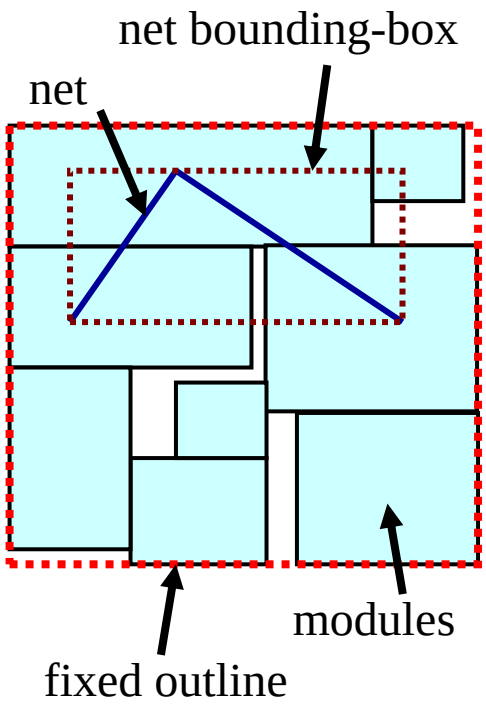


Perturbations & Solutions

- Perturbing B*-trees in simulated annealing
  - Op1: Rotate a module.
  - Op2: Flip a module.
  - Op3: Move a module to another place.
  - Op4: Swap two modules

Fixed-Outline Floorplanning

- Chen and Chang, "Modern floorplanning based on fast simulated annealing," ISPD'05 & TCAD'06
- Input: modules, netlist, fixed outline
- Output: module positions, orientations
- Objectives
  - Minimize the half-perimeter wirelength (HPWL)
  - All modules are within the fixed die (fixed-outline constraint) and no overlaps occur between modules



Fixed-Outline Constraint

- Given the total area A of modules, the percentage Γ of dead space, and the desired aspect ratio R^*, the outline is defined by

$$H^* = \sqrt{(1+\Gamma)AR^*} \qquad W^* = \sqrt{(1+\Gamma)A/R^*}$$

  – $R^* = H^*/W^*$, $H^*W^* = (1+\Gamma)A$

- Cost for floorplan F

$$\Phi(F) = \alpha A + \beta L + (1-\alpha-\beta)(R^*-R)^2$$

  – A: floorplan area
  – L: wirelength
  – R^*: fixed-outline aspect ratio
  – R: floorplan aspect ratio

Simulated Annealing Schedules

- Classical simulated annealing (SA)
  - Non-zero probability for up-hill move:

$$p = e^{-\Delta C/T}$$

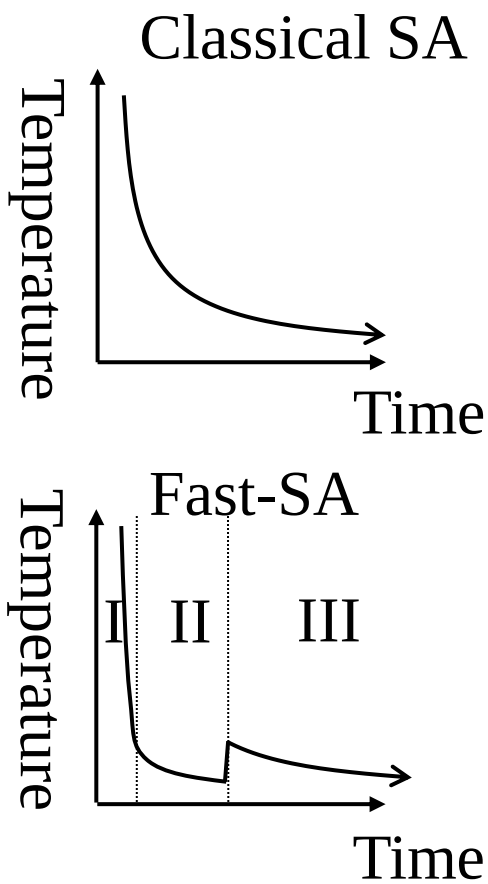
  - Initial temperature: $T = |\Delta_{avg} / \ln p|$, p is the initial acceptance rate (typically, close to 1.0), Δ_{avg} is the average cost of the up-hill moves
  - Classical temperature updating function: λ is set to a fixed value (e.g., 0.85)

$$T_{new} = \lambda T_{old}, \qquad 0 < \lambda < 1$$

- TimberWolf annealing schedule (Sechen and Sangiovanni-Vincentelli, DAC'86)
  - Increase λ gradually from its lowest value (0.8) to its highest value (approximately 0.95) and then gradually decreases λ back to its lowest value.

Fast Simulated Annealing

- Fast Simulated Annealing (Fast-SA) consists of 3 stages
  - High-temperature random search (temperature T -> a very large value)
  - Pseudo-greedy local search (T -> 0)
  - Hill-climbing search (increase T to simulate regular SA)



Fast Simulated Annealing (cont'd)

- Temperature update (T_1: initial temperature)

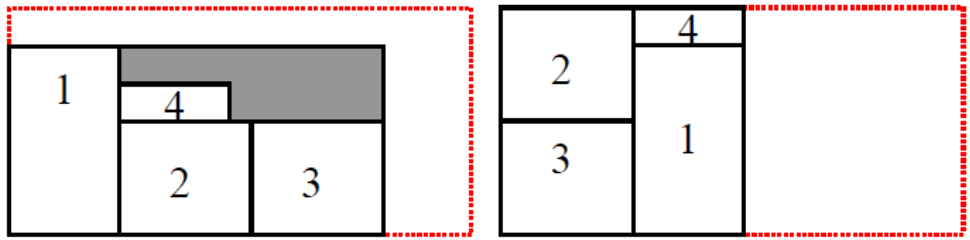
$$T_r = \begin{cases} \dfrac{\Delta avg}{\ln P} & r = 1 \\ \dfrac{T_1 \langle \Delta_{\cos t} \rangle}{rc} & 2 \le r \le k \\ \dfrac{T_1 \langle \Delta_{\cos t} \rangle}{r} & r > k \end{cases}$$

Δavg	Average uphill cost
P	Initial acceptance rate
$\langle \Delta_{cost} \rangle$	Average cost change for current temperature
r	Number of iterations
c, k	User-specified parameters (e.g., c = 100, k = 7)

- If $\langle \Delta_{cost} \rangle$ is larger, temperature decreases slowly.
- If $\langle \Delta_{cost} \rangle$ is smaller, temperature decreases quickly.

Adaptive Fast-SA

- The aspect ratio of the best floorplan area in the fixed outline is not the same as that of the outline.
- Decrease the weight of aspect ratio penalty (1 - α - β) to concentrate more on the floorplan wirelength/area optimization (i.e., increase α and β).
  - Adopt an adaptive method to control the weights in the cost function based on *n* most recent floorplans.
  - The more feasible floorplans, the less aspect ratio penalty.

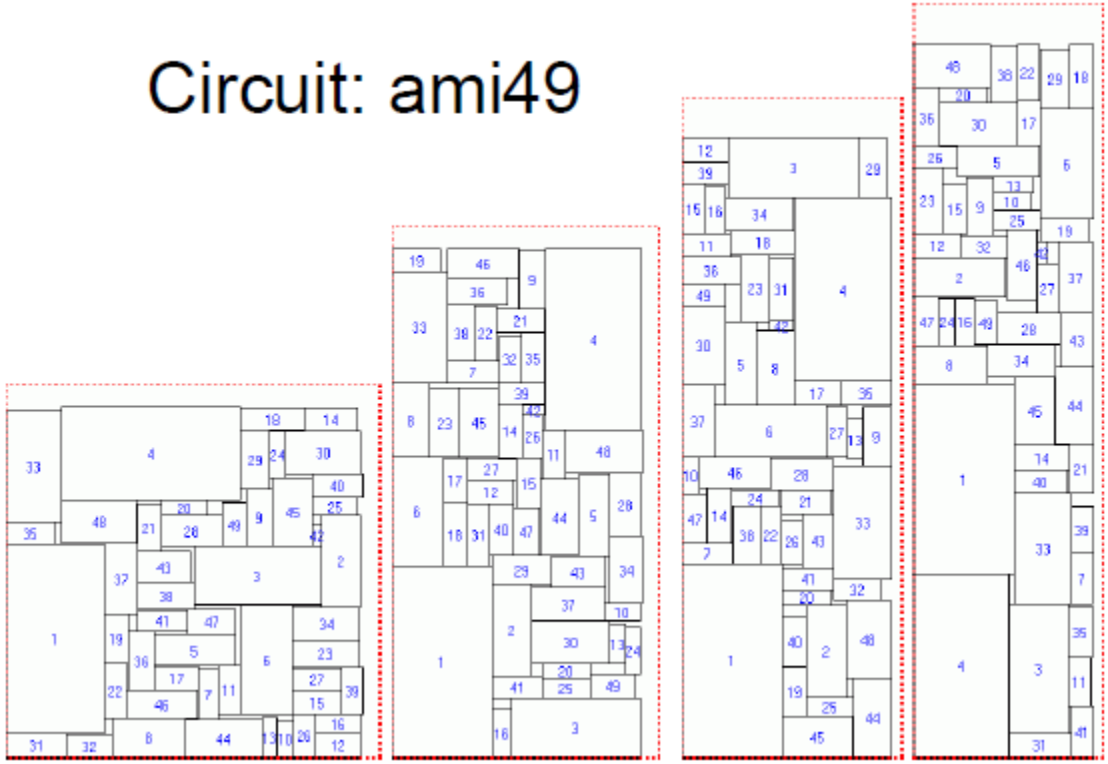


R = 0.5 R = 1

fixed-outline aspect ratio R* = 0.5

B*-tree Fixed-Outline Floorplanning Results

- B*-tree representation
- Fixed-outline floorplans with 10% dead space and aspect ratios 1, 2,3, and 4.



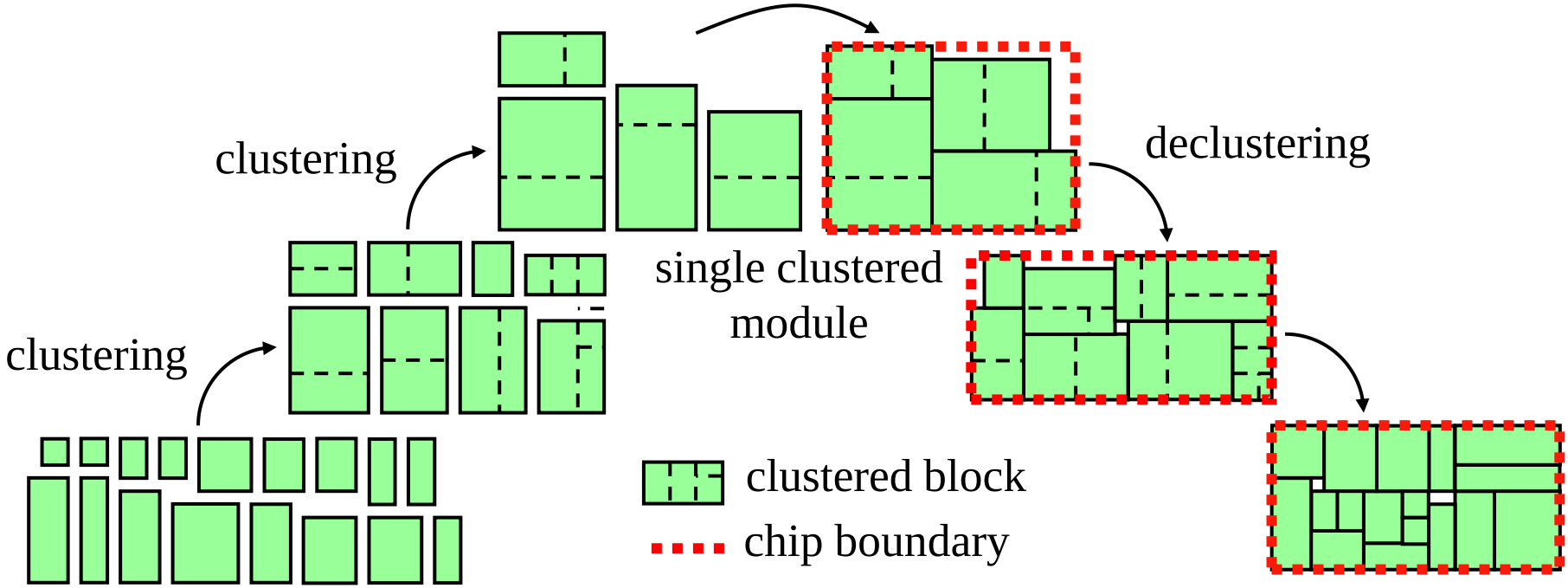
Floorplanning for Large-Scale Circuits

- Lee, Hsu, Chang, Yang, "Multilevel floorplanning/placement for large-scale modules using B*-trees," DAC'03 & TCAD'07
- Clustering (bottom-up coarsening) + declustering (top-down uncoarsening
  - Clustering
    - Iteratively groups a set of modules based on area utilization and module connectivity
    - Constructs a B*-tree to keep the geometric relations for the newly clustered modules
  - Declustering
    - Iteratively ungroups a set of the previously clustered modules (i.e., perform tree expansion)
    - Refines the solution using simulated annealing

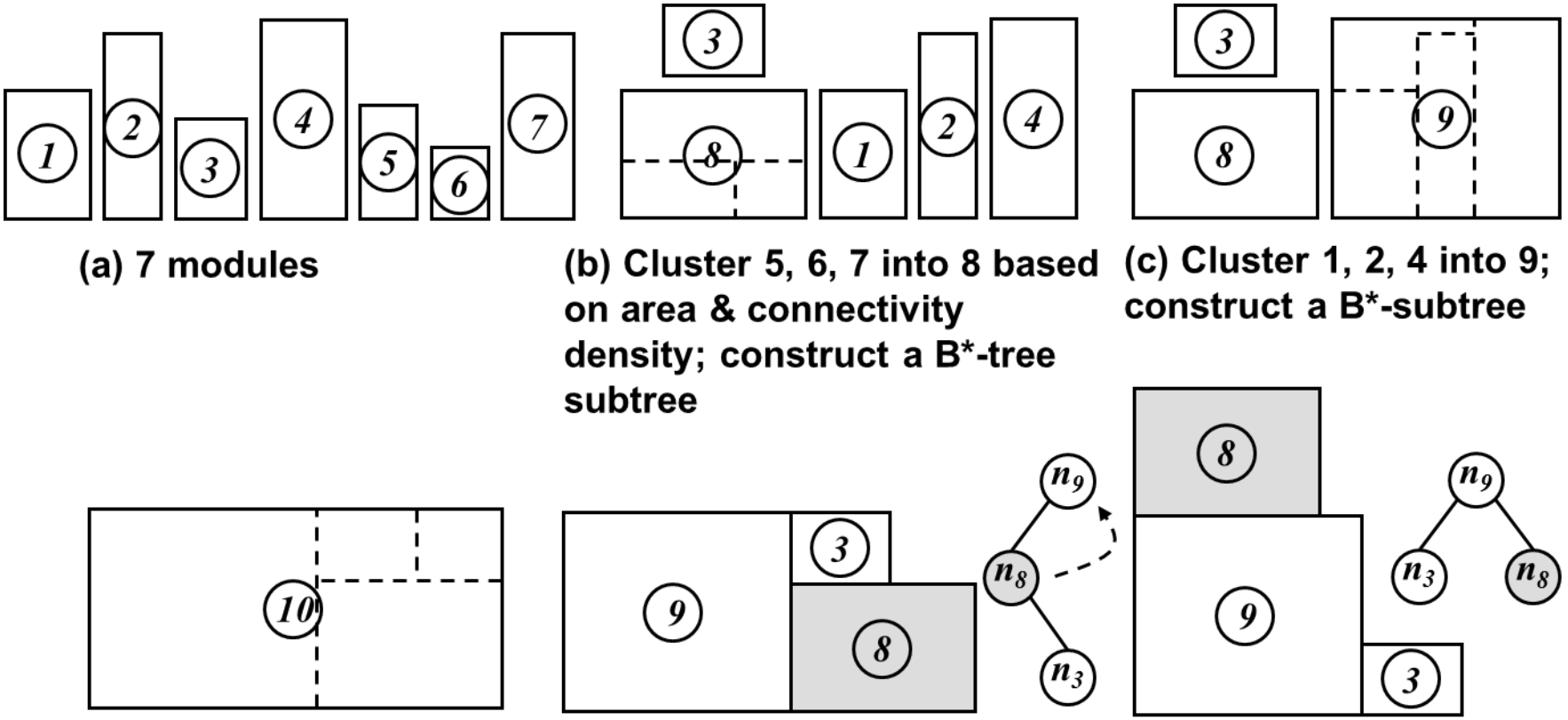
Λ-Shaped Multilevel Floorplanning

Cluster the modules based on area and local connectivity and create clustered modules for the next level.

Recursively decluster the clusters and use simulated annealing to refine the floorplan.



MB*-tree: Multilevel B*-tree



(a) 7 modules

(b) Cluster 5, 6, 7 into 8 based on area & connectivity density; construct a B*-tree subtree

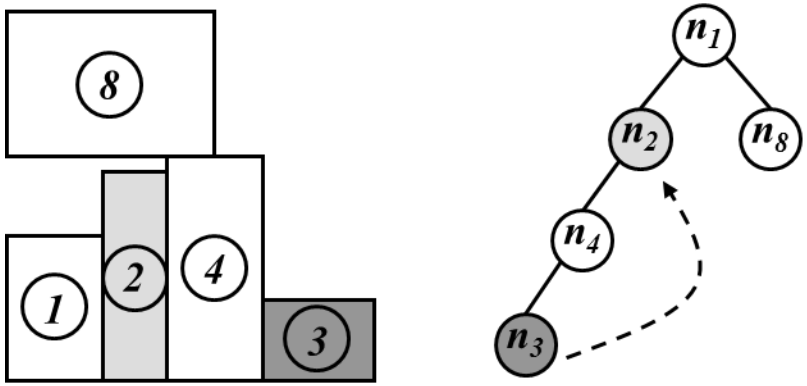
(c) Cluster 1, 2, 4 into 9; construct a B*-subtree

(d) Cluster 3, 8, 9 into 10; Construct a B*-subtree

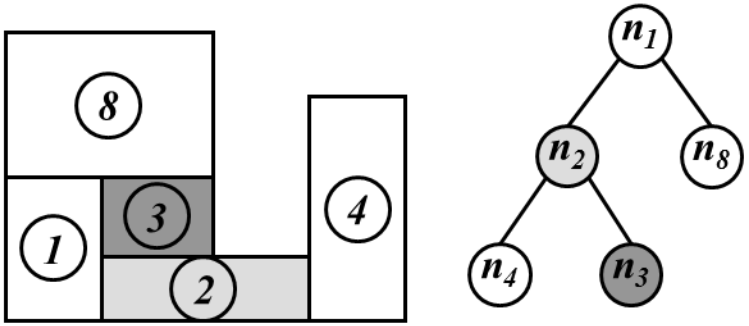
(e) Decluster 10 to 3, 8, 9

(f) Refine the solution by moving 8

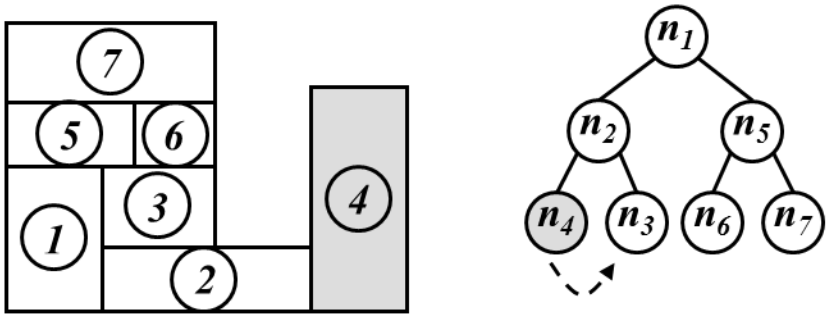
MB*-tree: Multilevel B*-tree (cont'd)



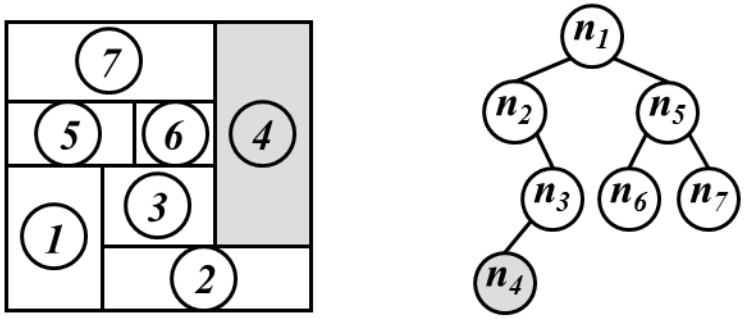
(g) Decluster 9 to 1, 2, 4



(h) Refine the solution by moving 2, 3



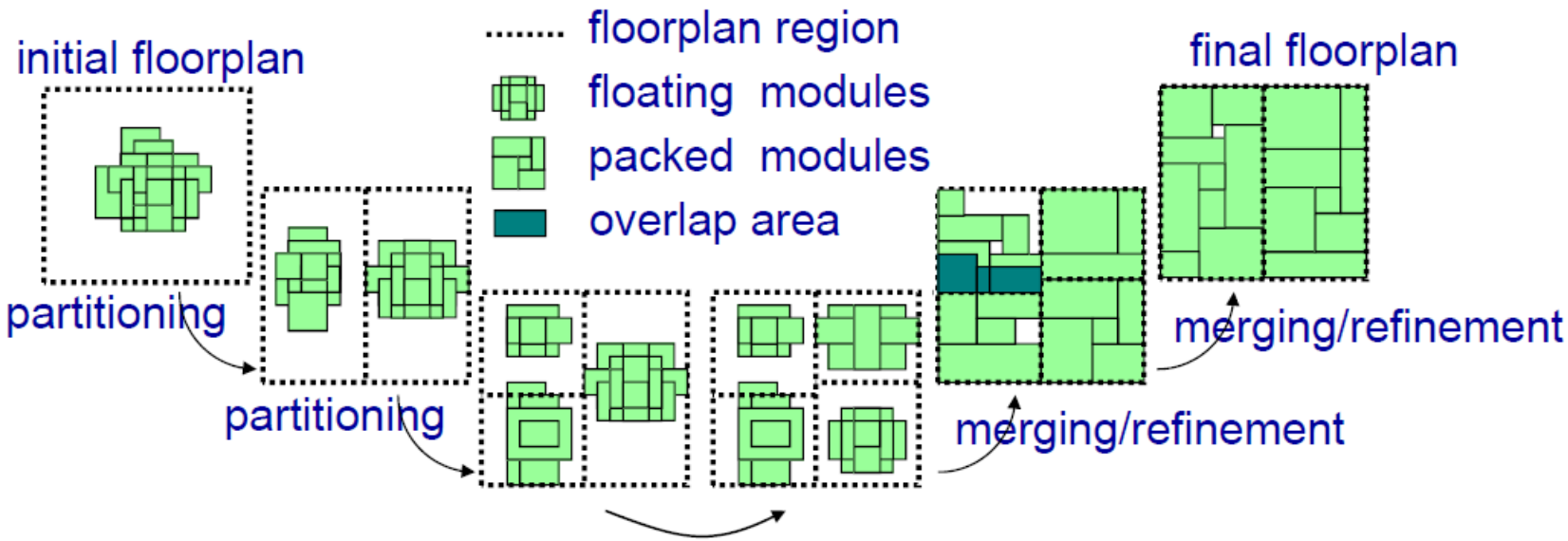
(i) Decluster 8 to 5, 6, 7



(j) Refine the solution by moving 4

IMF: V-Shaped Multilevel Floorplanning

- Chen, Chang, Lin, "IMF: interconnect-driven floorplanning for large-scale building-module designs," ICCAD'05

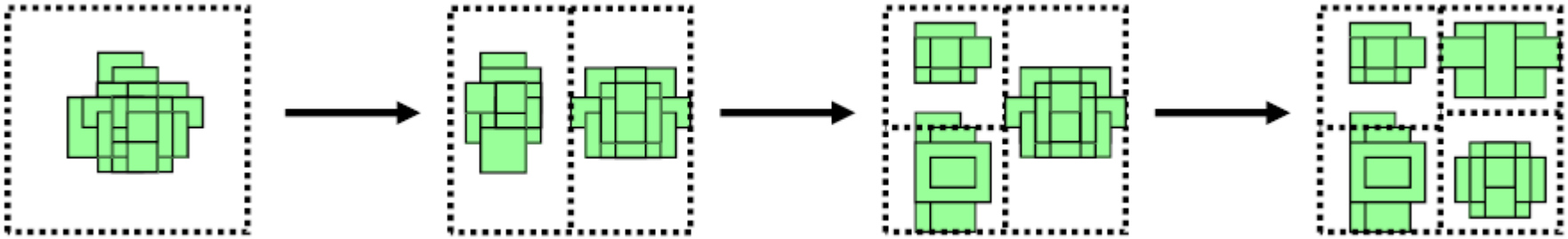


Perform partitioning to the circuit and determine the global locations of modules for the next level.

Use the flat floorplanner to pack the modules in the partitions and legalize/refine the solution.

Stage 1: Partitioning Stage

- All modules are set to the center of the chip region initially
- Partition the circuit recursively to minimize the interconnect and assign the regions of the modules
- The partitioning stage continues until the number of modules in each partition is smaller than a threshold, and the partitioned floorplan is obtained.

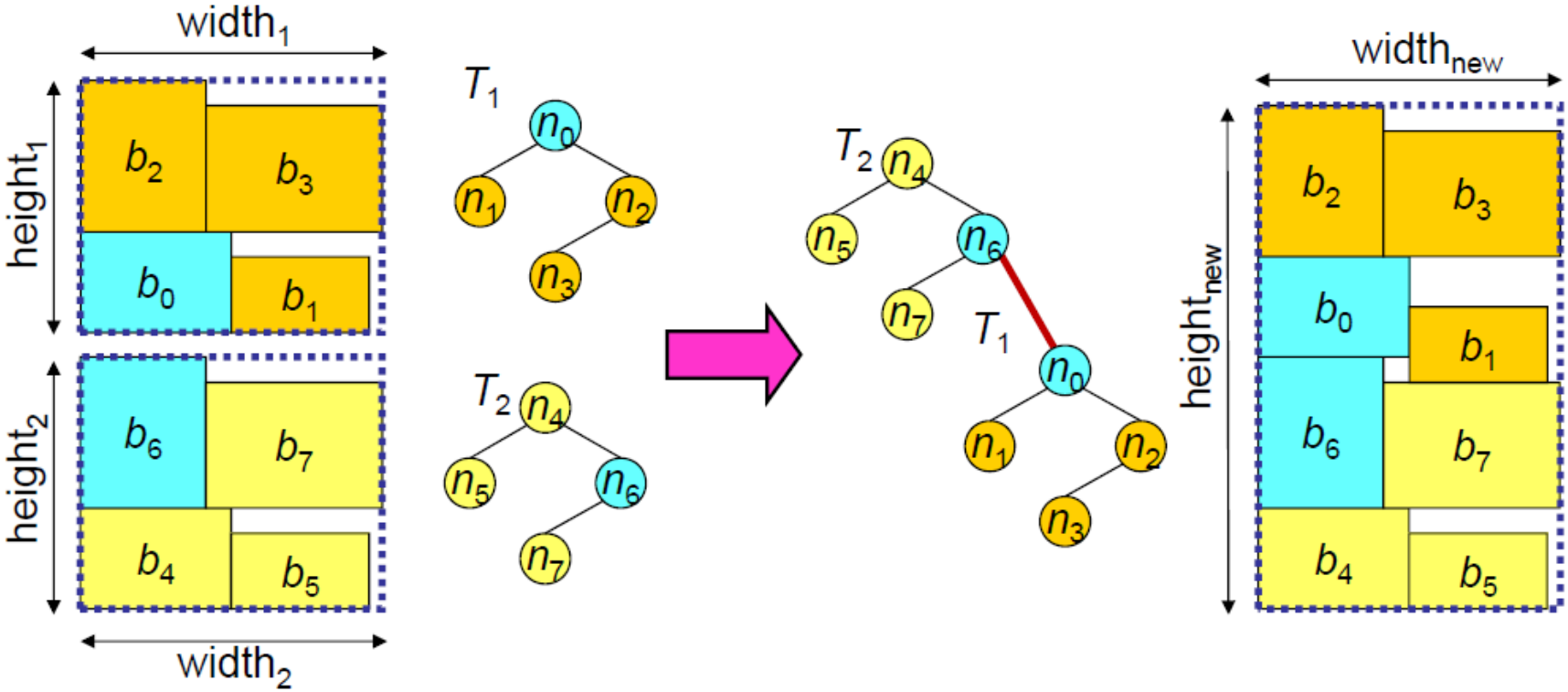


Stage 2: Merging Stage

- Construct a B*-tree and find the sub-floorplan for each sub-region (fixed-outline floorplanning)
- Cost function for the simulated annealing: area, wirelength, and aspect ratio penalty
- Merge two B*-trees (sub-floorplans) to form a new B*-tree (floorplan) recursively
- Refine the merged sub-floorplan using fixed-outline floorplanning again

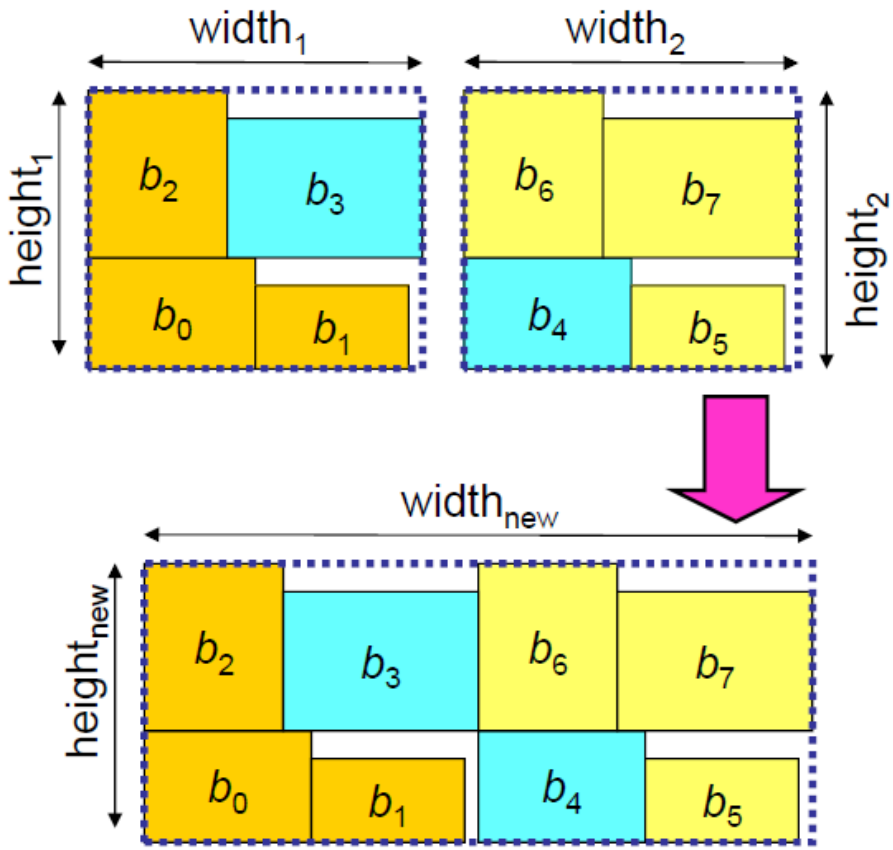
Vertical Merging

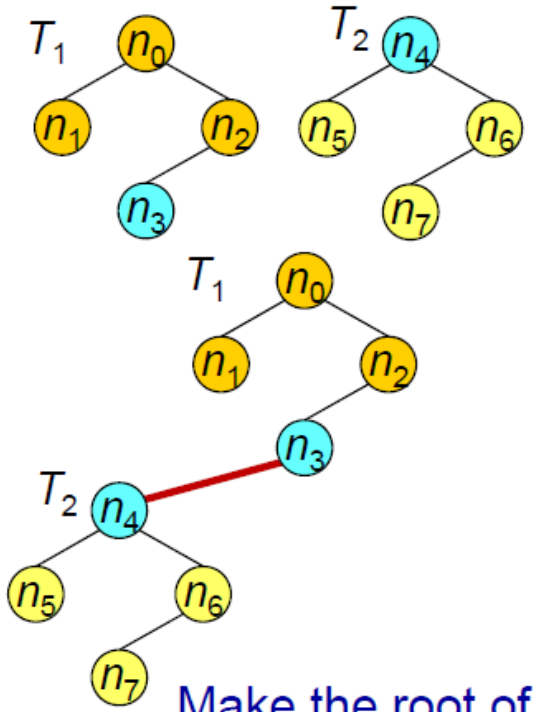
Make the root of the top B*-tree as the right child of the right-most node of the bottom B*-tree.



$$height_{new} \le height_1 + height_2$$
$$width_{new} = \max(\, width_1, width_2 \,)$$

Horizontal Merging



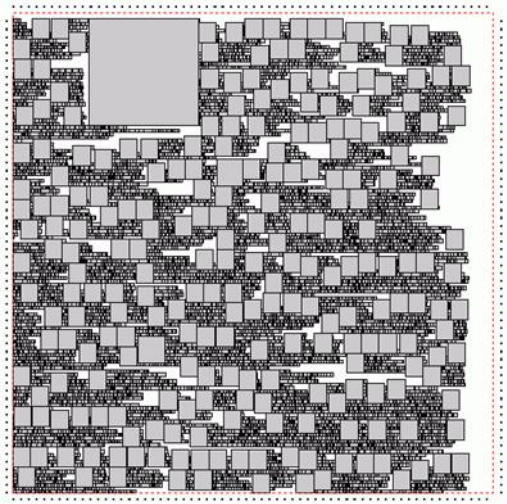


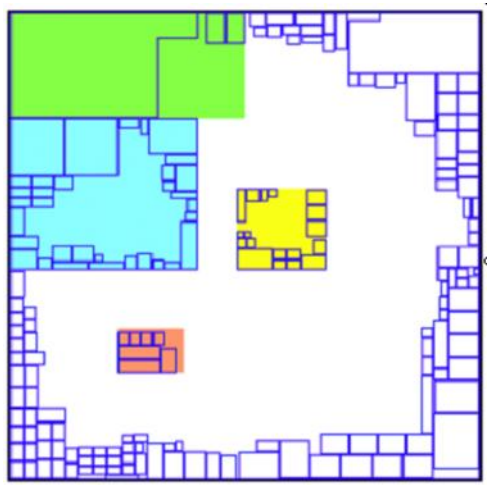
Make the root of the right B*-tree as the left child of the node corresponding to the right-most module of the left B*-tree.

$$height_{new} = \max(height_1, height_2)$$
$$width_{new} = width_1 + width_2$$

Macro Placement/Floorplanning

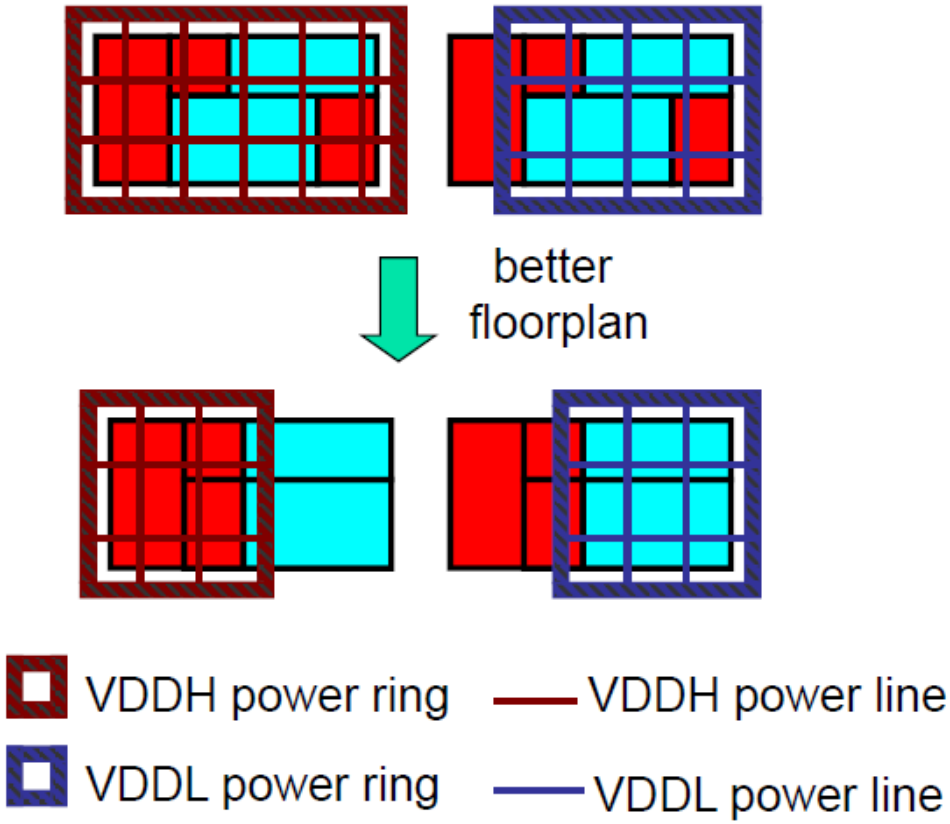
- Mixed sized cell/block placement/floorplanning: apply floorplanning techniques for macros to address various design constraints, e.g., range constraints, block rotation, block sizing

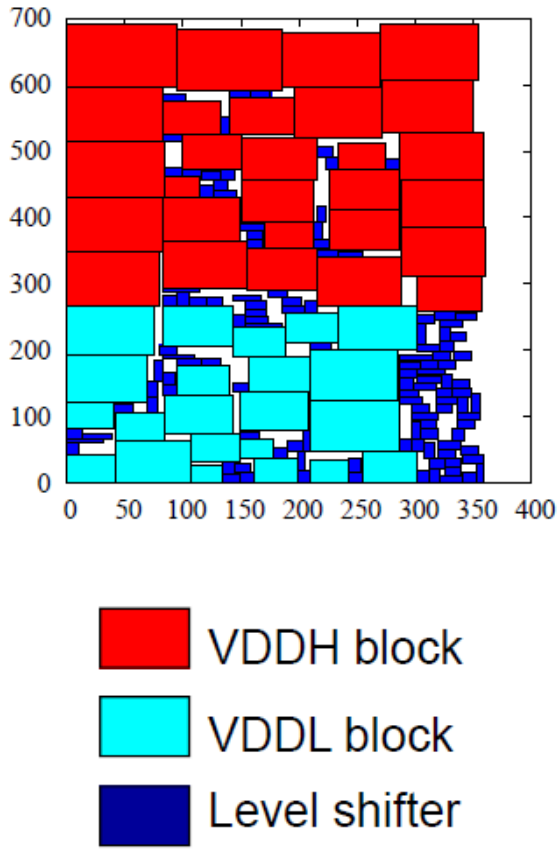




Voltage Island Aware Floorplanning

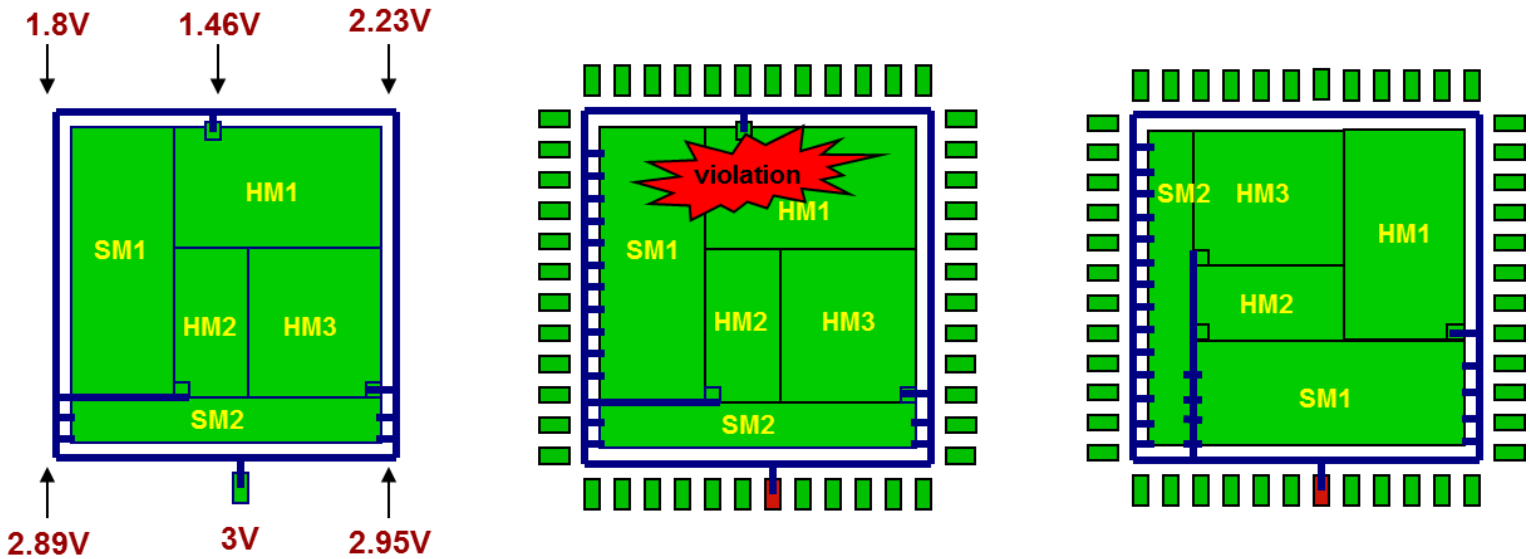
- Multiple supply voltages (voltage islands)





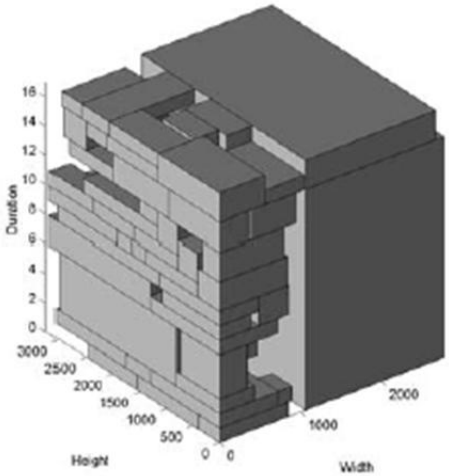
Voltage Drop Aware Floorplanning

- Power/ground networks for static/dynamic IR drop minimization (voltage drop aware floorplanning)



Beyond 2D Floorplanning

- Floorplanning for reconfigurable computing



- Floorplanning for digital microfluidic biochips
- SiP/2.5D/3D floorplanning

Existing Floorplan Representations

- Slicing: slicing tree, normalized Polished expression
- Mosaic: corner block list (ICCAD'00), twin binary tree (ISPD'01)
- Compacted: O-tree, B*-tree, corner sequence (TVLSI'03)
- General: sequence pair, bounded-sliceline grid (ICCAD'96), transitive closure graph (DAC'01), TCG-S (DAC'02), adjacent constraint graph (ICCD'04)

Comparison

Representation	Solution Space	Packing Time	Flexibility
Normalized Polish Expression	$O(n!2^{3n}/n^{1.5})$	$O(n)$	Slicing
Corner Block List	$O(n!2^{3n})$	$O(n)$	Mosaic
Twin Binary Sequence	$O(n!2^{3n}/n^{1.5})$	$O(n)$	Mosaic
O-tree	$O(n!2^{2n}/n^{1.5})$	$O(n)$	Compacted
B*-tree	$O(n!2^{2n}/n^{1.5})$	$O(n)$	Compacted
Corner Sequence	$\leq (n!)^2$	$O(n)$	Compacted
Sequence Pair	$(n!)^2$	$O(n^2)$	General
BSG	$O(n!C(n^2, n))$	$O(n^2)$	General
Transitive Closure Graph	$(n!)^2$	$O(n^2)$	General
TCG-S	$(n!)^2$	$O(n \lg n)$	General
Adjacent Constraint Graph	$O((n!)^2)$	$O(n^2)$	General

Existing Floorplanning Problems

- Outline free (variable die)
- Fixed outline (fixed die)
- Hard modules only
- Soft (and hard) modules
- Large scale
- Mixed size
- Pre-placed modules
- Range-constrained modules
- Boundary-constrained modules
- Abutment-constrained modules
- Symmetry-constrained modules
- Rectilinear modules
- Analog placement
- Beyond 2D

Existing Floorplanning Problems (cont'd)

- Co-synthesis with
  - Voltage islands
  - Power supply planning (voltage drop)
  - Interconnect planning
  - Bus planning
  - Buffer planning
  - Noise
  - SoC test scheduling
  - Micro-architecture pipelining
  - Double patterning